杂项

Morning Flower and Evening Oath

目录

第一章 Dancing Links **1**

1.1 精确覆盖（Dancing Links） . . . 1

1.2 最少行重复覆盖（Dancing Links） . . . 5

第二章 插头 DP **9**

2.1 插头 DP：网格单闭环计数 . . . 9

2.2 插头 DP：恰好 K 个不嵌套回路 . . . 14

第三章 动态 DP **18**

3.1 动态 DP：树上最大权独立集带修改 . . . 18

第四章 DP 优化 **22**

4.1 斜率优化 . . . 22

4.1.1 单调斜率凸包：最小值与决策 . . . 22

4.2 四边形不等式优化 . . . 25

4.2.1 定义与性质 . . . 25

4.2.2 分组 DP 的分治优化 . . . 25

4.2.3 分治决策单调性：一层分组 DP . . . 26

4.2.4 区间 DP 的 Knuth 优化 . . . 27

4.2.5 Knuth 优化：区间合并 DP . . . 29

4.3 WQS 二分 . . . 31

4.3.1 WQS 二分：至多 k 个不相邻位置 . . . 31

第五章 离线算法 **32**

5.1 CDQ 分治 . . . 32

5.1.1 静态三维偏序计数 . . . 32

5.1.2 CDQ 卷积递推：贡献只跨越一次 . . . 35

5.2 整体二分 . . . 35

5.2.1 整体二分：动态区间第 k 小 . . . 35

5.3 莫队算法 . . . 39

5.3.1 普通莫队算法 . . . 39

5.3.1.1 曼哈顿 MST 的递归 DFS 查询顺序 . . . 39

5.3.1.2 普通莫队：静态区间统计 . . . 39

5.3.2 带修改莫队 . . . 42

5.3.2.1 带修改的莫队 . . . 42

5.3.2.2 带修莫队：时间轴与区间状态 . . . 45

5.3.3 树上莫队 . . . 45

5.3.3.1 树上莫队：点权与边权 . . . 45

5.3.4 回滚莫队 . . . 47

5.3.4.1 回滚莫队 . . . 47

5.3.5 莫队二次离线 . . . 52

5.3.5.1 莫队二次离线：异或位数配对 . . . 52

5.3.5.2 莫队二次离线：前缀事件与自身扣除 . . . 56

第六章 分数规划 **57**
6.1 0/1 分数规划：Dinkelbach 57
6.2 0/1 分数规划 57

第七章 表达式求值 **58**
7.1 整数表达式求值 58

第八章 珂朵莉树/颜色段均摊 **61**
8.1 珂朵莉树（颜色段均摊） 61

第一部分 跨类依赖 **62**

附录 A 数论 **63**
A.1 模算术简介 63
A.1.1 静态模整数 63

附录 B 并查集 **67**
B.1 并查集 67
B.1.1 并查集 67

附录 C 树状数组 **69**
C.1 树状数组第 k 小与组合依赖 69

附录 D 线段树 **71**
D.1 线段树基础 71
D.1.1 通用线段树与左右边界搜索 71

附录 E 树上问题 **76**
E.1 树链剖分 76
E.1.1 重链剖分与 LCA 76

附录 F 最短路问题 **82**
F.1 最短路 82
F.1.1 SPFA：队列松弛、负环判定与有限路径 82

附录 G 生成树问题 **85**
G.1 最小生成树 85
G.1.1 曼哈顿最小生成树 85
G.1.2 Kruskal 最小生成树与生成森林 88

第一章　Dancing Links

1.1　精确覆盖（Dancing Links）

问题描述：给定 m 列和若干行（每行是一组列号，即稀疏 0/1 矩阵），选出若干行，使每一列**恰好**被一个选中行覆盖；求任意一组解的行号，或判定无解。

核心思路：Knuth 的 X 算法加舞蹈链：所有 1 连成十字循环链表，0 号节点是列头链表的表头，`s[c]` 是列 c 当前剩余的候选行数。`cover(c)` 把列 c 从列头链表摘除，并把含有列 c 的每一行的其余节点从各自列中纵向摘除（这些行与之后选的行冲突）；`uncover` 按相反顺序原样接回。`dfs` 每次选候选行最少的列，依次尝试其中每一行：记录行号，覆盖该行涉及的其余列后递归，失败则恢复并换下一行。列头链表为空即找到解；成功返回前同样恢复链表，所以可以重复求解。

接口契约：ExactCover(m,rows)：列 0..m-1，各稀疏行内列号互异、无需排序；行可重复或为空，保留原 0-based 行号。solve() 返回 optional<vector<int»，无解为 nullopt，m=0 为有值的空解。只求任意首解，不保证最少行数。成功/失败均恢复链与列计数，可重复调用及复制；实现字段不可外改，移后源对象仅销毁或重新赋值。m 和行数 <INT_MAX，1+m+ 非零数 ≤INT_MAX。构造 O(行数 +m+ 非零数)，空间 O(m+ 非零数)，递归深度 ≤m，搜索最坏指数。

```
struct ExactCover
{
    struct Node { int l, r, u, d, c, row; };
    int m;
    vector<Node> a;
    vector<int> s, ans;

    ExactCover(int columns, const vector<vector<int>> &rows) : m(columns)
    {
        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
        size_t z = size_t(m) + 1;
        for (const auto &r : rows)
        {
            assert(r.size() <= size_t(INT_MAX) - z);
            z += r.size();
        }
        a.reserve(z);
        s.assign(m + 1, 0);
        for (int c = 0; c <= m; c++)
            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});
```

稀疏行建链与删除（接上页同一结构体）：

```
        vector<int> seen(m, -1);
        for (int i = 0; i < int(rows.size()); i++)
        {
            int h = -1;
            for (int col : rows[i])
            {
                assert(0 <= col && col < m && seen[col] != i);
                seen[col] = i;
                int c = col + 1, x = a.size(), u = a[c].u;
                a.push_back({x, x, u, c, c, i});
                a[u].d = a[c].u = x;
                s[c]++;
                if (h == -1) h = x;
                else
                {
                    int l = a[h].l;
                    a[x].l = l; a[x].r = h;
                    a[l].r = a[h].l = x;
                }
            }
        }
    }

    void cover(int c)
    {
        a[a[c].l].r = a[c].r;
        a[a[c].r].l = a[c].l;
        for (int i = a[c].d; i != c; i = a[i].d)
            for (int j = a[i].r; j != i; j = a[j].r)
            {
                a[a[j].u].d = a[j].d;
                a[a[j].d].u = a[j].u;
                s[a[j].c]--;
            }
    }
```

逆序恢复与搜索（接上页同一结构体）：

```
    void uncover(int c)
    {
        for (int i = a[c].u; i != c; i = a[i].u)
            for (int j = a[i].l; j != i; j = a[j].l)
            {
                s[a[j].c]++;
                a[a[j].u].d = a[a[j].d].u = j;
            }
        a[a[c].l].r = a[a[c].r].l = c;
    }

    bool dfs()
    {
        int c = a[0].r;
        if (!c) return true;
        for (int j = a[c].r; j; j = a[j].r)
            if (s[j] < s[c]) c = j;
        cover(c);
        for (int i = a[c].d; i != c; i = a[i].d)
        {
            ans.push_back(a[i].row);
            for (int j = a[i].r; j != i; j = a[j].r) cover(a[j].c);
            bool ok = dfs();
            for (int j = a[i].l; j != i; j = a[j].l) uncover(a[j].c);
            if (ok)
            {
                uncover(c);
                return true;
            }
            ans.pop_back();
        }
        uncover(c);
        return false;
    }

    optional<vector<int>> solve()
    {
        ans.clear();
        ans.reserve(m);
        if (!dfs()) return nullopt;
        return ans;
    }
};
```

模板题使用：Luogu P4929 / 精确覆盖

题意：读入 N×M 的 01 矩阵，N,M≤500 且非零数 ≤5000；每列恰被选择行覆盖一次。稀疏建模后输出原 1-based 行号，任一可行解均可；无解输出 No Solution!。

所需模板：ExactCover（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4929

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> rows(n);
    for (int i = 0; i < n; i++)
        for (int j = 0, x; j < m; j++)
        {
            cin >> x;
            if (x) rows[i].push_back(j);
        }
    ExactCover dlx(m, rows);
    auto ans = dlx.solve();
    if (!ans) cout << "No Solution!\n";
    else
    {
        for (int i : *ans) cout << i + 1 << ' ';
        cout << '\n';
    }
}
```

1.2 最少行重复覆盖（Dancing Links）

问题描述：给定 m 列和若干行（每行是一组列号），选出**最少**的行，使每一列至少被一个选中行覆盖（允许重复覆盖），返回一组最优解的行号或判定无解。

核心思路：舞蹈链上的分支限界搜索。重复覆盖时行永远不会作废，只需横向摘除：`remove(x)` 把 `x` 所在列的其他节点（含列头）从各自的行链表中摘下，这一列就从待覆盖列表中消失，而各列的候选行数 `s` 保持不变。`dfs` 选候选最少的未覆盖列，枚举其中每一行：摘除该行覆盖的所有列，递归后逆序 `restore`。剪枝用 `lower_bound`：贪心选出若干两两不能被同一行同时覆盖的未覆盖列，其个数是至少还需的行数；当前深度加此下界不小于已知最优 `best` 时剪掉。`best` 初值为 $m+1$，到达全部覆盖时更新答案。

接口契约：MinimumCover(m, rows)，列号 0..m-1，每行内部互异，允许空行/重复行。solve() 返回最少行数的一组原 0-based 行号；nullopt 无解，有值空向量表示 m=0。每列至少一次，允许重叠；不含权值、覆盖次数或全解枚举。0≤m<INT_MAX，行数 <INT_MAX，m+1+ 非零数 ≤INT_MAX；前提由调用方保证。构造 O(行数 +m+ 非零数)，空间 O(m+ 非零数)；递归搜索最坏指数、深度 ≤m，每状态 O(m+ 非零数)。成功失败均恢复链，可重复求解或复制；重建用重新赋值。字段及辅助方法不作为可修改接口，移动后源对象只销毁或赋值。下界选取不能被同一行共同覆盖的列，至少还需同样多行；按最少候选列搜索，逆序恢复只改左右链。

```
struct MinimumCover
{
    struct Node
    {
        int l, r, u, d, c, row;
    };
    int m, best = 0;
    vector<Node> a;
    vector<int> s, path, ans;
    vector<unsigned char> mark;

    MinimumCover(int columns, const vector<vector<int>> &rows) : m(columns)
    {
        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
        size_t z = size_t(m) + 1;
        for (const auto &r : rows)
        {
            assert(r.size() <= size_t(INT_MAX) - z);
            z += r.size();
        }
        a.reserve(z);
        s.assign(m + 1, 0);
        mark.resize(m + 1);
        path.reserve(m);
        ans.reserve(m);
        for (int c = 0; c <= m; c++)
            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, -1});
```

稀疏建链与删列（接上页同一结构体）：

```
        vector<int> seen(m, -1);
        for (int i = 0; i < int(rows.size()); i++)
        {
            int h = -1;
            for (int col : rows[i])
            {
                assert(0 <= col && col < m && seen[col] != i);
                seen[col] = i;
                int c = col + 1, x = a.size(), u = a[c].u;
                a.push_back({x, x, u, c, c, i});
                a[u].d = x;
                a[c].u = x;
                s[c]++;
                if (h == -1) h = x;
                else
                {
                    int l = a[h].l;
                    a[x].l = l;
                    a[x].r = h;
                    a[l].r = x;
                    a[h].l = x;
                }
            }
        }
    }

    // x is a selected row node, not a column header.
    void remove(int x)
    {
        for (int i = a[x].d; i != x; i = a[i].d)
        {
            a[a[i].l].r = a[i].r;
            a[a[i].r].l = a[i].l;
        }
    }
```

逆序恢复与可采纳下界（接上页同一结构体）：

```
    void restore(int x)
    {
        for (int i = a[x].u; i != x; i = a[i].u)
        {
            a[a[i].l].r = i;
            a[a[i].r].l = i;
        }
    }

    int lower_bound()
    {
        fill(mark.begin(), mark.end(), 0);
        for (int c = a[0].r; c; c = a[c].r) mark[c] = 1;
        int bound = 0;
        for (int c = a[0].r; c; c = a[c].r)
        {
            if (!mark[c]) continue;
            bound++;
            mark[c] = 0;
            for (int i = a[c].d; i != c; i = a[i].d)
                for (int j = a[i].r; j != i; j = a[j].r)
                    mark[a[j].c] = 0;
        }
        return bound;
    }
```

最优搜索与求解入口（接上页同一结构体）：

```
    void dfs()
    {
        int depth = path.size(), c = a[0].r;
        if (!c)
        {
            if (depth < best)
            {
                best = depth;
                ans = path;
            }
            return;
        }
        if (lower_bound() >= best - depth) return;
        for (int j = a[c].r; j; j = a[j].r)
            if (s[j] < s[c]) c = j;
        for (int i = a[c].d; i != c; i = a[i].d)
        {
            path.push_back(a[i].row);
            remove(i);
            for (int j = a[i].r; j != i; j = a[j].r) remove(j);
            dfs();
            for (int j = a[i].l; j != i; j = a[j].l) restore(j);
            restore(i);
            path.pop_back();
        }
    }

    optional<vector<int>> solve()
    {
        path.clear();
        ans.clear();
        best = m + 1;
        dfs();
        if (best > m) return nullopt;
        return ans;
    }
};
```

第二章　插头 DP

2.1　插头 DP：网格单闭环计数

问题描述：给定由 . 和 * 组成的矩形网格，. 是必须经过的格子，* 不能经过，只能在上下左右相邻的格子间移动。求有多少种方法，用**恰好一条**闭合回路经过所有 .，且每个 . 恰好经过一次；回路不区分起点和行走方向。例如四个格子都是 . 的 2×2 网格，答案为 1。

核心思路：按行逐格扫描。已处理部分是若干条尚未闭合的路径，只需记录它们穿过当前扫描轮廓（已处理格的下边与当前格左侧）的位置，以及哪两个端点属于同一条路径：轮廓上第 j 个槽用 4 位编号，同编号的两个槽是同一路径的两端，0 表示无插头。处理格 (i, j) 时，槽 j、$j+1$ 分别是来自左侧、上方的插头，转移后变为向下、向右的插头。每个 . 度数恰为 2，根据左、上插头决定新建、延伸、合并路径或闭环；目标只有一个覆盖全部格子的环，所以只在最后一个 . 且没有其他插头时允许闭环计数。每行开始时整体左移一槽，右边界插头恒为 0。

思路中的情况	代码中的处理	行
取出左、上插头	解码状态，`x`、`y` 为左、上插头编号，并把两槽清零	44
遇到 *	不能有来自左、上的插头，原样保留状态，跳过该格	61
左、上都没有插头	下、右两格都可用时新建一条路径，向下、向右各一个端点（临时编号 15）	65
恰有一个插头	把这条路径向下或向右延伸（两种选择分别计数）	74
两个插头属于不同路径	合并两条路径：把编号 `y` 全部改成 `x`	88
两个插头属于同一路径	形成闭环；只有在最后一个 . 且其余槽全为 0 时计入答案	94
不同编号表示相同连接关系	`add` 按首次出现重新编号后再存入哈希表，避免重复状态	48
换行	所有状态左移一槽（4 位），对齐下一行的轮廓	33

接口契约：grid_cycle<Count>(a) 统计覆盖所有可用格子的单个无向简单环；矩形网格中. 必须经过、* 不能经过，仅上下左右相邻。空网格/全障碍返回 0。自动转置到较窄方向，宽度 m≤15，尺寸须可用 int 表示。每个未闭合连通块是一条路径，在轮廓上恰有两个同编号端点；宽度 ≤15 时最多 8 条路径，临时编号 15 不与规范编号冲突。每个边集唯一对应逐格选择，不按起点或方向重复计数。

Count 支持 0/1 构造和 +=；固定整数必须保证所有中间计数不溢出，可改模整数或 cpp_int。S 为最多可达状态数，固定宽度计数的期望时间 O(n*m^2*S)、状态表 O(S)，另有网格 O(nm)。S 随宽度指数增长；cpp_int 还需计入位复杂度。

```cpp
template <class Count>
Count grid_cycle(vector<string> a)
{
    int n = a.size();
    if (n == 0) return Count(0);
    int m = a[0].size();
    for (auto &row : a)
    {
        assert((int)row.size() == m);
        for (char c : row) assert(c == '.' || c == '*');
    }
    if (m == 0) return Count(0);
    if (n < m)
    {
        vector<string> b(m, string(n, '*'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) b[j][i] = a[i][j];
        a.swap(b);
        swap(n, m);
    }
    assert(m <= 15);
    pair<int, int> last = {-1, -1};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] == '.') last = {i, j};
    using Key = unsigned long long;
    unordered_map<Key, Count> dp, next;
    dp[0] = Count(1);
    Count answer = 0;
    for (int i = 0; i < n; i++)
    {
        next.clear();
        for (auto &[s, ways] : dp) next[s << 4] += ways;
        dp.swap(next);
```

轮廓转移：解码左、上插头，重编号，障碍格与新建路径（接上页同一函数）：

```
        for (int j = 0; j < m; j++)
        {
            next.clear();
            bool down = i + 1 < n && a[i + 1][j] == '.';
            bool right = j + 1 < m && a[i][j + 1] == '.';
            for (auto &[s, ways] : dp)
            {
                vector<int> p(m + 1);
                for (int k = 0; k <= m; k++) p[k] = s >> (4 * k) & 15;
                int x = p[j];
                int y = p[j + 1];
                p[j] = 0;
                p[j + 1] = 0;
                auto add = [&]()
                {
                    array<int, 16> id{};
                    int count = 0;
                    Key key = 0;
                    for (int k = 0; k <= m; k++)
                    {
                        int v = p[k];
                        if (v && !id[v]) id[v] = ++count;
                        key |= Key(id[v]) << (4 * k);
                    }
                    next[key] += ways;
                };
                if (a[i][j] == '*')
                {
                    if (!x && !y) add();
                }
                else if (!x && !y)
                {
                    if (down && right)
                    {
                        p[j] = 15;
                        p[j + 1] = 15;
                        add();
                    }
                }
```

延伸、合并与最终闭环（接上页同一函数）：

```
                else if (!x || !y)
                {
                    if (down)
                    {
                        p[j] = x + y;
                        add();
                        p[j] = 0;
                    }
                    if (right)
                    {
                        p[j + 1] = x + y;
                        add();
                    }
                }
                else if (x != y)
                {
                    for (int &v : p)
                        if (v == y) v = x;
                    add();
                }
                else if (make_pair(i, j) == last)
                {
                    if (all_of(p.begin(), p.end(), [](int v)
                    {
                        return v == 0;
                    })) answer += ways;
                }
            }
            dp.swap(next);
        }
    }
    return answer;
}
```

数值与验证 固定整数要保证所有中间计数可表示，不能只检查最后答案。P5056 仅内部 (n-1)(m-1) 个格子可能二选一，其余转移至多一个。所有中间计数不超过 2 的 121 次方，因此下面用 128 位整数仍能保证精确计数。已穷举 74954 个小障碍网格并与独立递归环搜索核对；完整 2 至 14 阶偶数方格与 OEIS A003763 核对。输入超过题目范围时状态数仍可能很大。

模板题使用：P5056 插头 DP

题意：2≤n,m≤12，所有点格必须形成唯一闭环。仅同时存在下、右邻格时可能二选一，至多 $(12\text{-}1)^2=121$ 次，因此全部中间计数 ≤2^121，用 ___int128 安全；十进制输出。

所需模板：grid_cycle（第 9 页，2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5056

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (auto &row : a) cin >> row;
    __int128 answer = grid_cycle<__int128>(a);
    string out;
    do
    {
        out += char('0' + answer % 10);
        answer /= 10;
    } while (answer);
    reverse(out.begin(), out.end());
    cout << out << '\n';
}
```

2.2 插头 DP：恰好 K 个不嵌套回路

问题描述：给定由 . 和 * 组成的网格，只能在上下左右相邻格之间走，求用**恰好** k **个**闭合回路经过所有 . 且每格恰好一次的方案数（回路不区分起点和方向）；默认要求任何回路不包含另一回路，也可允许嵌套。

核心思路：与单回路插头 DP 一样逐格扫描（宽大于高时先转置），轮廓每槽 4 位存路径编号，同编号两槽为同一条未闭合路径的两端，0 表示无插头；状态再按已闭合的环数 c 分开存于 `dp[c]`。每格取出左插头 `x`、上插头 `y` 分情况转移，`add` 规范化编号后存入目标环数的哈希表。左、上为同一路径的两端时闭合成环，环数加一；不允许嵌套时要求当前格左边的插头数为偶数，否则新环落在另一条路径内部。扫完所有格后取 `dp[k]` 中的空状态。

思路中的情况	代码中的处理	行
换行	所有状态左移一槽（4 位），对齐下一行的轮廓	37
取出左、上插头	解码状态，`x`、`y` 为左、上插头编号，并把两槽清零	51
遇到 *	不能有插头进入，原样保留状态	68
左、上都没有插头	下、右两格都可用时新建一条路径（临时编号 15）	72
恰有一个插头	把这条路径向下或向右延伸，两种选择分别计数	81
两个插头属于不同路径	合并两条路径：把编号 `y` 全部改成 `x`	95
两个插头属于同一路径	闭合成环，环数加一；禁止嵌套时检查左侧插头数的奇偶	101

接口契约：grid_cycles<Count>(a,k,nested=false) 统计覆盖全部. 格子的恰好 k 个无向简单环；* 为障碍，只有上下左右相邻。默认禁止任意环包含另一个环；nested=true 允许嵌套。无可用格且 k=0 时返回 1，其余无法覆盖时返回 0。k 为非负 int 且 k<INT_MAX；矩形尺寸可用 int 表示，自动转置后宽度 m≤15。

同编号的两个插头表示同一路径端点，规范化编号。闭环时环数加 1；禁止嵌套时检查当前格左侧插头数为偶数。每个环在其最后扫描顶点闭合，该奇偶性是环的嵌套深度奇偶；若存在嵌套，必有深度 1 的环被拒绝。只在扫完全部格子、无剩余插头时取答案。

Count 需支持 0/1 构造、默认构造为 0、复制和 +=；所有中间计数均须可表示，可用模整数或 cpp_int。S 为固定环数下最多轮廓状态数，固定宽度计数下期望时间 O(n*m^2*(k+1)*(S+1))，两层状态表 O((k+1)*(S+1))，网格 O(nm)。S 随宽度指数增长；高精度计数另计位复杂度。

```cpp
template <class Count>
Count grid_cycles(vector<string> a, int k, bool nested = false)
{
    assert(k >= 0);
    int n = a.size();
    if (n == 0) return Count(k == 0);
    int m = a[0].size();
    long long cells = 0;
    for (auto &row : a)
    {
        assert((int)row.size() == m);
        for (char c : row)
        {
            assert(c == '.' || c == '*');
            cells += c == '.';
        }
    }
    if (cells == 0) return Count(k == 0);
    if (k == 0 || k > cells / 4) return Count(0);
    if (n < m)
    {
        vector<string> b(m, string(n, '*'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) b[j][i] = a[i][j];
        a.swap(b);
        swap(n, m);
    }
    assert(m <= 15);
    using Key = unsigned long long;
    vector<unordered_map<Key, Count>> dp(k + 1), next(k + 1);
    dp[0][0] = Count(1);
```

行移位、规范化与创建路径（接上页同一函数）：

```
    for (int i = 0; i < n; i++)
    {
        for (int c = 0; c <= k; c++)
        {
            next[c].clear();
            for (auto &[s, ways] : dp[c]) next[c][s << 4] += ways;
        }
        dp.swap(next);
        for (int j = 0; j < m; j++)
        {
            for (auto &t : next) t.clear();
            bool down = i + 1 < n && a[i + 1][j] == '.';
            bool right = j + 1 < m && a[i][j + 1] == '.';
            for (int c = 0; c <= k; c++)
            {
                for (auto &[s, ways] : dp[c])
                {
                    array<int, 16> p{};
                    for (int t = 0; t <= m; t++) p[t] = s >> (4 * t) & 15;
                    int x = p[j];
                    int y = p[j + 1];
                    p[j] = 0;
                    p[j + 1] = 0;
                    auto add = [&](int count)
                    {
                        array<int, 16> id{};
                        int top = 0;
                        Key key = 0;
                        for (int t = 0; t <= m; t++)
                        {
                            int v = p[t];
                            if (v && !id[v]) id[v] = ++top;
                            key |= Key(id[v]) << (4 * t);
                        }
                        next[count][key] += ways;
                    };
                    if (a[i][j] == '*')
                    {
                        if (!x && !y) add(c);
                    }
                    else if (!x && !y)
                    {
                        if (down && right)
                        {
                            p[j] = 15;
                            p[j + 1] = 15;
                            add(c);
                        }
                    }
```

延伸、合并与闭环计数（接上页同一函数）：

```
                    else if (!x || !y)
                    {
                        if (down)
                        {
                            p[j] = x + y;
                            add(c);
                            p[j] = 0;
                        }
                        if (right)
                        {
                            p[j + 1] = x + y;
                            add(c);
                        }
                    }
                    else if (x != y)
                    {
                        for (int &v : p)
                            if (v == y) v = x;
                        add(c);
                    }
                    else if (c < k)
                    {
                        int parity = 0;
                        for (int t = 0; t < j; t++) parity ^= p[t] != 0;
                        if (nested || parity == 0) add(c + 1);
                    }
                }
            }
            dp.swap(next);
        }
    }
    auto it = dp[k].find(0);
    return it == dp[k].end() ? Count(0) : it->second;
}
```

第三章　动态 DP

3.1　动态 DP：树上最大权独立集带修改

问题描述：给定点带权的树，支持修改单点权值，每次修改后求全树的最大权独立集（所选点两两不相邻，可为空集）。

核心思路：设 f_0, f_1 为不选、选 u 时子树的最优值，只把轻儿子的贡献汇总为 $g_0 = \sum \max(f_0, f_1)$、$g_1 = w_u + \sum f_0$，则 u 的 (f_0, f_1) 等于 $(\max, +)$ 矩阵 $[[g_0, g_0], [g_1, -\infty]]$ 乘重儿子的向量。线段树按 `dfn` 存各点矩阵，把重链从链顶到链底连乘，再作用于 $(0, -\infty)$ 就得到链顶的 DP 值（`chain`）。修改时更新本点矩阵并沿重链上跳：每到一条链，求出链顶新旧 DP 值，用其差修正链顶父亲的 g_0, g_1，再更新父亲的矩阵；根链链顶的 $\max(f_0, f_1)$ 即答案。

代码依赖：HLD（第 76 页，E.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：DynamicDP(h,w) 接收已 build 的 HLD h（点号 1..n，根任意，须比本对象活得久）与点权 w[1..n]；set(u,val) 修改点权，answer() 返回全树最大权独立集（允许空集）。每个点维护轻儿子贡献 g0=∑max(f0,f1)、g1=w+∑f0，对应 (max,+) 矩阵 [[g0,g0],[g1,-inf]]；线段树按 dfn 维护矩阵积，重链自顶向下连乘，再乘 (0,-inf) 得链顶 (f0,f1)。修改时沿重链上跳，用链顶新旧值之差更新父亲的 g0、g1。构造 O(n)，每次修改 O($\log^2$ n)，查询 O(log n)；HLD 的 DFS 为递归。换成其他树形 DP：把转移写成 f_u 等于 M_u 乘 f_ 重儿子的半环矩阵乘法，矩阵只依赖 u 自身与轻儿子信息即可套用。

```
// Dynamic DP: max-weight independent set of a tree under point weight changes (P4719).
// Vertices 1..n, built HLD (any root). f0/f1 = best in subtree with u excluded/included.
struct DynamicDP
{
    using ll = long long;
    static constexpr ll NEG = LLONG_MIN / 4;
    struct M
    {
        ll a[2][2];
    };

    // (max,+) product; matrices along a heavy chain multiply from top to bottom.
    static M mul(const M &x, const M &y)
    {
        M r;
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++) r.a[i][j] = max(x.a[i][0] + y.a[0][j], x.a[i][1] + y.a[1][j]);
        return r;
    }

    const HLD &h;
    int n;
    vector<ll> w, g0, g1; // g0 = sum max(f0,f1), g1 = w + sum f0, over light children only
    vector<int> ed;       // ed[top] = dfn of the chain's last vertex
    vector<M> t;

    M leaf(int u) const { return {{{g0[u], g0[u]}, {g1[u], NEG}}}; }

    void build(int p, int l, int r)
    {
        if (l == r)
```

```
    {
        t[p] = leaf(h.rk[l]);
        return;
    }
    int m = (l + r) / 2;
    build(2 * p, l, m), build(2 * p + 1, m + 1, r);
    t[p] = mul(t[2 * p], t[2 * p + 1]);
}

void update(int p, int l, int r, int x)
{
    if (l == r)
    {
        t[p] = leaf(h.rk[l]);
        return;
    }
    int m = (l + r) / 2;
    x <= m ? update(2 * p, l, m, x) : update(2 * p + 1, m + 1, r, x);
    t[p] = mul(t[2 * p], t[2 * p + 1]);
}

M query(int p, int l, int r, int ql, int qr) const
{
    if (ql <= l && r <= qr) return t[p];
    int m = (l + r) / 2;
    if (qr <= m) return query(2 * p, l, m, ql, qr);
    if (ql > m) return query(2 * p + 1, m + 1, r, ql, qr);
    return mul(query(2 * p, l, m, ql, qr), query(2 * p + 1, m + 1, r, ql, qr));
}

// (f0, f1) of a chain top: chain product applied to the empty vector (0, -inf).
pair<ll, ll> chain(int top) const
{
    M p = query(1, 1, n, h.dfn[top], ed[top]);
    return {p.a[0][0], p.a[1][0]};
}

DynamicDP(const HLD &h, const vector<ll> &weight) : h(h), n(h.n), w(weight), g0(n + 1), g1(n + 1), ed(n +
 1), t(4 * n + 4)
{
    vector<ll> f0(n + 1), f1(n + 1);
    for (int i = n; i >= 1; i--)
    {
        int u = h.rk[i];
        f1[u] += w[u], g1[u] += w[u];
        for (int v : h.g[u])
            if (v != h.fa[u])
            {
                f0[u] += max(f0[v], f1[v]), f1[u] += f0[v];
                if (v != h.son[u]) g0[u] += max(f0[v], f1[v]), g1[u] += f0[v];
            }
    }
    for (int i = 1; i <= n; i++) ed[h.top[h.rk[i]]] = i;
    build(1, 1, n);
}

```

```
    // Set w[u] = val in O(log^2 n).
    void set(int u, ll val)
    {
        g1[u] += val - w[u], w[u] = val;
        while (true)
        {
            int top = h.top[u];
            auto old = chain(top);
            update(1, 1, n, h.dfn[u]);
            auto now = chain(top);
            if (h.fa[top] == top) break; // root chain
            int p = h.fa[top];
            g0[p] += max(now.first, now.second) - max(old.first, old.second);
            g1[p] += now.first - old.first;
            u = p;
        }
    }

    // Maximum weight of an independent set in the whole tree.
    ll answer() const
    {
        auto [a, b] = chain(h.rk[1]);
        return max(a, b);
    }
};
```

模板题使用：Luogu P4719 【模板】动态 DP

题意：n,m≤10^5 的树，每次把一个点的权改为 y，输出全树最大权独立集。以 1 为根建 HLD 后构造 DynamicDP，每次 set 后输出 answer。

所需模板：HLD（第 76 页，E.1.1 节）、DynamicDP（第 18 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4719

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<long long> w(n + 1);
    for (int i = 1; i <= n; i++) scanf("%lld", &w[i]);
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        h.add(u, v);
    }
    h.build(1);
    DynamicDP d(h, w);
    while (m--)
    {
        int x;
        long long y;
        scanf("%d%lld", &x, &y);
        d.set(x, y);
        printf("%lld\n", d.answer());
    }
}
```

第四章　DP 优化

4.1　斜率优化

4.1.1　单调斜率凸包：最小值与决策

功能：维护直线集合 $y = mx + b$：直线按斜率非增加入、查询点 x 非减时，求所有直线在 x 处的最小值及取到它的直线编号，用于斜率优化 DP。

核心思路：下凸壳存于数组 `q`，有效部分从 `head` 开始，斜率依次递减，每条直线在一段连续的 x 上最优。加线时同斜率只保留截距更小者；若队尾直线 b 与前一条 a 的交点不在 b 与新线 c 的交点左侧，b 永远不会最优，弹出（交点比较用 int128 交叉相乘，不做除法）。查询 x 单调增大，最优直线只会往斜率更小的一侧移动，`query` 在下一条直线严格更优时推进 `head`，被越过的直线以后也不会再最优。

接口契约：默认空；add(m,b,id) 按斜率非增加入直线 mx+b，query(x) 按 x 非减查询，返回 (pair<int128,int>)(最小值, 编号)。首次查询前至少加一条线；可交替加线与查询。并列保留最早加入的直线，不是最小 id；同斜率只留最小截距，截距也同则留旧线。q/head 仅供只读；查询会永久淘汰旧候选，不支持回退 x 或任意斜率。要求 |m|,|b|,|x|<=10^18，总加线数 <INT_MAX。先提升 int128 再做差与交叉乘法，范围内精确无浮点；返回值勿未经证明窄化为 long long。总 M 次加线、Q 次查询 O(M+Q)，空间 O(M)，被查询淘汰的前缀仍留在 vector 中。

```
struct MonotoneHull
{
    using ll = long long;
    using I = __int128_t;
    struct Line
    {
        ll m, b;
        int id;

        I value(ll x) const
        {
            return I(m) * x + b;
        }
    };
    vector<Line> q;
    int head = 0;
    ll last_x = 0;
    bool queried = false;

    void add(ll m, ll b, int id)
    {
        assert(q.empty() || m <= q.back().m);
        if (int(q.size()) > head && q.back().m == m)
        {
            if (q.back().b <= b) return;
            q.pop_back();
        }
        Line c{m, b, id};
        while (int(q.size()) - head >= 2)
        {
            auto a = q[q.size() - 2];
            auto b = q.back();
            if ((I(b.b) - a.b) * (b.m - I(c.m)) <
                (I(c.b) - b.b) * (a.m - I(b.m))) break;
            q.pop_back();
        }
        q.push_back(c);
    }
```

查询（接上页同一结构体）：

```
    pair<I, int> query(ll x)
    {
        assert(int(q.size()) > head);
        assert(!queried || last_x <= x);
        queried = true;
        last_x = x;
        while (head + 1 < int(q.size()) && q[head + 1].value(x) < q[head].value(x))
        {
            head++;
        }
        return {q[head].value(x), q[head].id};
    }
};
```

4.2 四边形不等式优化

4.2.1 定义与性质

本节代价 $w(l,r)$ 只对闭区间 $1 \le l \le r \le n$ 定义，DP 求**最小值**；求最大值时把下列不等号全部反向。

四边形不等式 对所有 $a \le b \le c \le d$，

$$w(a,c) + w(b,d) \le w(a,d) + w(b,c),$$

即“两段交叉区间的代价和”不超过“外层区间加内层区间的代价和”。把 $M_{l,r} = w(l,r)$ 看作上三角矩阵，这正是 **Monge 性质**：任取两行 $a < b$、两列 $c < d$（且 $b \le c$），主对角线之和不超过副对角线之和。**只需验证相邻格**：若对所有 $i+1 \le j$ 有 $w(i,j) + w(i+1,j+1) \le w(i,j+1) + w(i+1,j)$，把 $a \le i < b$、$c \le j < d$ 的这些不等式全部相加，中间项相消即得一般形式（$b \le c$ 保证每格都有定义）。相邻形式等价于“右端扩一格的增量 $w(l,r+1) - w(l,r)$ 随左端 l 右移而不增”。

区间包含单调性 对所有 $a \le b \le c \le d$，$w(b,c) \le w(a,d)$，即大区间代价不小于其子区间。它与四边形不等式**互不蕴含**：$w(l,r) = -[l=1]$ 使四边形不等式取等号，却有 $w(2,3) = 0 > w(1,4) = -1$；$w(l,r) = [l<r]$ 满足包含单调性，却有 $w(1,2) + w(2,3) = 2 > w(1,3) + w(2,2) = 1$。

最优决策单调性 它是 **DP 的最优转移点**的性质，不是代价函数的性质：某一转移把 i 的最优决策记为 $opt(i)$，若 $i < i'$ 时 $opt(i) \le opt(i')$，称决策单调。多个决策并列最优时，opt 必须按统一规则取（本节一律取**最小**的最优决策），否则单调性可能不成立，代码也不能按单调性剪枝。以下两种 DP 形式分别说明：四边形不等式（及 Knuth 情形下的包含单调性）**推出**哪一种决策单调性；没有这些条件时决策也可能单调，但必须另行证明。

常见满足条件的代价 非负权的区间和 $w = \sum_{i=l}^{r} x_i$：四边形不等式取等号且包含单调。$w = g(S_r - S_{l-1})$，其中 S 为非降前缀和、g 凸：满足四边形不等式；g 再非降则包含单调。区间内所有无序对的非负权和 $w = \sum_{l \le i < j \le r} e_{ij}$：右减左恰为 $[a, b-1]$ 与 $[c+1, d]$ 之间的点对权，两条件都满足。满足条件的代价的非负线性组合仍满足相应条件。

4.2.2 分组 DP 的分治优化

转移形式 把 $1..n$ 分成恰好 K 个非空连续段，段 $[l,r]$ 的代价为 $w(l,r)$，最小化总代价。令 $f_g(i)$ 表示前 i 个元素恰分成 g 段的最小代价，j 为前一段的结束位置：

$$f_g(i) = \min_{g-1 \le j < i} \big\{ f_{g-1}(j) + w(j+1, i) \big\}, \qquad g \le i \le n.$$

初值 $f_0(0) = 0$，$f_0(i) = +\infty$ $(i > 0)$；$i < g$ 时 $f_g(i) = +\infty$（不可达），答案 $f_K(n)$。

条件与结论 若 w 满足四边形不等式，则对**任意**给定的上一层 f_{g-1}，取最小最优决策 $opt_g(i)$ 时有 $opt_g(i) \le opt_g(i+1)$。不需要包含单调性。

证明. 设 $i < i'$，$j = opt_g(i)$，若 $j' = opt_g(i') < j$。则 j' 对 i 合法、j 对 i' 合法（$j < i < i'$），且两者的 f_{g-1} 有限。取 $(a,b,c,d) = (j'+1, j+1, i, i')$，四边形不等式给出

$$\big[f_{g-1}(j) + w(j+1,i')\big] - \big[f_{g-1}(j') + w(j'+1,i')\big] \le \big[f_{g-1}(j) + w(j+1,i)\big] - \big[f_{g-1}(j') + w(j'+1,i)\big] < 0,$$

最后的严格号来自“j 是 i 的**最小**最优决策而 $j' < j$”。于是 j 对 i' 严格优于 j'，矛盾。□

分治 求 f_g 在 $[l,r]$ 上的值且已知决策落在 $[lo, hi]$ 时，取中点 mid，枚举 $j \in [lo, \min(hi, mid-1)]$ 求出 $opt(mid)$（严格小于才更新，得最小最优决策）；左半递归用 $[lo, opt(mid)]$，右半用 $[opt(mid), hi]$。由上述结论，左右两半的最小最优决策确实落在这两个范围内。同一递归深度各区间的枚举范围首尾相接，总量 $O(n)$，深度 $O(\log n)$，所以一层 $O(n \log n)$、K 层 $O(Kn \log n)$（w 须 $O(1)$ 求值）。

对应接口 `monotone_dp_layer(prev,first,cost,opt)` 计算一层：`prev` 即 $f_{g-1}(0..n)$，`first`= g，`cost(j+1,i)` 即 $w(j+1,i)$，返回 $f_g(0..n)$，`opt[i]` 即 $opt_g(i)$；$i < g$ 时返回 `inf`、`opt` 为 -1。`prev[first-1]`= $f_{g-1}(g-1)$ 必须有限，保证每个 $i \geq g$ 至少有决策 $j = g-1$。分治要求整层 `prev` 事先算完，**不能**直接用于依赖同层未计算状态的转移，例如不限段数的 $f(i) = \min_j\{f(j) + w(j+1,i)\}$：中点处的 $f(j)$ 尚未求出。这类转移需改用 CDQ 分治套分治、单调队列加二分等其他写法。

4.2.3 分治决策单调性：一层分组 DP

问题描述：把 $1..n$ 分成恰好 K 个非空连续段、段代价 $w(l,r)$ 满足四边形不等式时，求最小总代价；本函数完成其中一层转移 $f_g(i) = \min_j\{f_{g-1}(j) + w(j+1,i)\}$。

核心思路：上节已证明最小最优决策 $opt_g(i)$ 随 i 非减。先算中点的最优决策，左半只在不超过它的决策中找、右半只在不小于它的决策中找，每层递归总枚举 $O(n)$。

接口契约：monotone_dp_layer(prev,first,cost,opt) 返回 cur，prev 下标 0..n，1<=first<=n<INT_MAX。对 i=first..n 求 min(prev[j]+cost(j+1,i))，j 在 first-1..i-1，cost 是闭区间代价；用于第 g 组时 first=g。inf=LLONG_MAX/4；prev[first-1] 必须可达，其他 inf 跳过，cost 对全部合法区间有限。所有有限 prev、cost 及转移和须严格在 (-inf,inf)，保证 long long 加法安全。cur[0..first-1]=inf，opt 对应为-1；其余 opt 返回最小最优 j。调用方须证明该并列规则下 opt[i] 非减，本函数不会检查，任意 DP 不能套用。cost 须无副作用且 O(1) 才能得 O(n log(n+1)) 单层时间；递归深度 O(log n)，输出与辅助 O(n)。

```cpp
template<class Cost>
vector<long long> monotone_dp_layer(const vector<long long> &prev, int first,
                                    Cost cost, vector<int> &opt)
{
    using ll = long long;
    constexpr ll inf = LLONG_MAX / 4;
    int n = int(prev.size()) - 1;
    assert(1 <= first && first <= n && prev[first - 1] != inf);
    vector<ll> cur(n + 1, inf);
    opt.assign(n + 1, -1);
    auto solve = [&](auto &&self, int l, int r, int lo, int hi) -> void
    {
        if (l > r) return;
        int mid = l + (r - l) / 2;
        for (int j = lo; j <= min(hi, mid - 1); j++)
        {
            if (prev[j] == inf) continue;
            ll value = prev[j] + cost(j + 1, mid);
            if (value < cur[mid])
            {
                cur[mid] = value;
                opt[mid] = j;
            }
        }
        assert(opt[mid] != -1);
        self(self, l, mid - 1, lo, opt[mid]);
        self(self, mid + 1, r, opt[mid], hi);
    };
    solve(solve, first, n, first - 1, n - 1);
    return cur;
}
```

模板题使用：P4767 [IOI 2000] 邮局加强版

题意：n≤3000 个村庄、k≤300 个邮局，坐标 1..10000，最小化各村到最近邮局距离和。排序后每个邮局服务一段连续村庄，邮局放在段中位数；f_g(i)=min f_{g-1}(j)+w(j+1,i)，w 由前缀和 O(1) 求。左端每左移一格的增量 x[(a+r)/2 下取整]-x[a-1] 随 r 增大不减，故 w 满足四边形不等式，逐层调用 monotone_dp_layer，第 g 层 first=g；总 O(kn log n)。

所需模板：monotone_dp_layer（第 26 页，4.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4767

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<long long> x(n + 1), sum(n + 1);
    for (int i = 1; i <= n; i++) cin >> x[i];
    sort(x.begin() + 1, x.end());
    for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + x[i];
    auto cost = [&](int l, int r) -> long long
    {
        int mid = (l + r) / 2;
        return x[mid] * (mid - l + 1) - (sum[mid] - sum[l - 1]) +
               (sum[r] - sum[mid]) - x[mid] * (r - mid);
    };
    vector<long long> dp(n + 1, LLONG_MAX / 4);
    vector<int> opt;
    dp[0] = 0;
    for (int g = 1; g <= k; g++)
    {
        dp = monotone_dp_layer(dp, g, cost, opt);
    }
    cout << dp[n] << '\n';
}
```

4.2.4 区间 DP 的 Knuth 优化

转移形式 典型如石子合并：

$$f(l,r) = w(l,r) + \min_{l\le k<r}\{f(l,k)+f(k+1,r)\}\ (l<r), \qquad f(i,i)=0.$$

$w(l,r)$ 只在 $l<r$ 时使用，$w(i,i)$ 无意义。若题目给叶子代价 $f(i,i)=e_i$，每个叶子恰出现一次，可先按 0 计算再加 $\sum_{i=l}^{r} e_i$。记 $opt(l,r)$ 为最小最优分割点，约定 $opt(i,i)=i$。

充分条件 对长度至少为 2 的区间，w 同时满足**四边形不等式**与**区间包含单调性**：

$$w(a,c)+w(b,d)\le w(a,d)+w(b,c), \qquad w(b,c)\le w(a,d), \qquad a\le b\le c\le d.$$

结论：$opt(l,r-1)\le opt(l,r)\le opt(l+1,r)$（$r-l\ge 2$），于是只枚举 $k\in[opt(l,r-1),opt(l+1,r)]$。

引理（f 也满足四边形不等式） 对 $d-a$ 归纳证 $f(a,c)+f(b,d)\le f(a,d)+f(b,c)$。$a=b$ 或 $c=d$ 时两边相同。否则 $a<b$、$c<d$，令 $z=opt(a,d)$。若 $b=c$：当 $z<c$ 时，$f(a,c)\le w(a,c)+f(a,z)+f(z+1,c)$，对较短的 $[z+1,d]$ 用归纳得 $f(z+1,c)+f(c,d)\le f(z+1,d)+f(c,c)$，再用 $w(a,c)\le w(a,d)$ 即得 $f(a,c)+f(c,d)\le f(a,d)+f(c,c)$；

$z \geq c$ 对称（拆 $f(c,d)$，对 $[a,z]$ 归纳，用 $w(c,d) \leq w(a,d)$）。若 $b < c$：令 $y = opt(b,c)$。当 $z \leq y$ 时，用 z 拆 $f(a,c)$、用 y 拆 $f(b,d)$，由 w 的四边形不等式与归纳式 $f(z+1,c)+f(y+1,d) \leq f(z+1,d)+f(y+1,c)$ 相加即得；当 $z > y$ 时，用 y 拆 $f(a,c)$、用 z 拆 $f(b,d)$，配合归纳式 $f(a,y)+f(b,z) \leq f(a,z)+f(b,y)$ 同理。□（包含单调性只在 $b=c$ 一步使用，前面的反例 $w=-[l=1]$ 正好在这里失效。）

决策范围的推导 记 $g_k(l,r) = f(l,k)+f(k+1,r)$，比较两个分割点时 $w(l,r)$ 抵消。*左界*：对 $k < k' \leq r-2$，引理用于 $(k+1,k'+1,r-1,r)$ 得 $g_k(l,r)-g_{k'}(l,r) \geq g_k(l,r-1)-g_{k'}(l,r-1)$。取 $k' = K = opt(l,r-1)$，对任意 $k < K$ 右边 > 0（K 是最小最优），故 k 不是 (l,r) 的最优，$opt(l,r) \geq K$。*右界*：令 $K = opt(l+1,r) \geq l+1$，对 $k' > K$，引理用于 $(l,l+1,K,k')$ 得 $g_K(l,r)-g_{k'}(l,r) \leq g_K(l+1,r)-g_{k'}(l+1,r) \leq 0$，所以 K 不劣于更大的分割点，最小最优决策 $\leq K$。左界用到严格不等号、右界只用到非严格不等号，两者在“取最小最优决策”下同时成立；若改为都取最大最优决策也成立，但不能混用。

复杂度 按区间长度递增计算。固定长度 L 时，各 l 的枚举量 $\sum_l \big(opt(l+1,r)-opt(l,r-1)+1\big)$ 相邻项相消，不超过 $2n$；共 n 种长度，时间 $O(n^2)$，表 $O(n^2)$，比朴素 $O(n^3)$ 少一个 n。长度为 2 时 $[opt(l,l), opt(l+1,l+1)] = [l,l+1]$，须截到 $r-1=l$。

与分组 DP 分治优化的区别

	分组 DP 分治优化	区间 DP Knuth 优化
转移	$f_g(i)$ 只依赖已算完的上一层	$f(l,r)$ 依赖同一张表的更短区间
代价条件	仅四边形不等式	四边形不等式 + 区间包含单调性
决策关系	单侧：$opt_g(i) \leq opt_g(i+1)$	双侧：$opt(l,r-1) \leq opt(l,r) \leq opt(l+1,r)$
证明对象	只用 w，与上一层取值无关	先证 f 满足四边形不等式
实现与复杂度	中点分治，每层 $O(n \log n)$	按长度递推，总 $O(n^2)$

反例 $n=4$、$w(l,r) = -[l=1]$：四边形不等式取等号但不满足包含单调性。真值 $f(1,4) = -3$（$k=3$），而 $opt(1,3)=2$、$opt(2,4)=2$，Knuth 只枚举 $k=2$ 得 -2。环形问题（如环形石子合并）复制成长 $2n-1$ 的链，答案取 $\min_i f(i,i+n-1)$。

4.2.5 Knuth 优化：区间合并 DP

问题描述： 计算区间合并型 DP $f(l,r)=w(l,r)+\min_{l\le k<r}\{f(l,k)+f(k+1,r)\}$，$f(i,i)=0$，其中 w 满足四边形不等式与区间包含单调性（如石子合并）。

核心思路： 按区间长度递增计算；由上节的双侧决策关系，(l,r) 的最小最优分割点只在 $[opt(l,r-1),opt(l+1,r)]$ 内枚举，同一长度的枚举量相消为 $O(n)$，总 $O(n^2)$。

接口契约： knuth_interval_dp(n,cost,opt) 返回 f，下标 1..n：f[i][i]=0，l<r 时 f[l][r]=cost(l,r)+min(f[l][k]+f[k+1][r])，k 取 l..r-1；其余位置为 inf=LLONG_MAX/4。opt[i][i]=i，l<r 时 opt[l][r] 为最小最优 k，只在 [opt[l][r-1],opt[l+1][r]] 内枚举。调用方须证明对长度 ≥2 的区间 cost 满足四边形不等式与区间包含单调性（见上文），本函数不检查。1≤n，n+1 个 vector 各 n+1 项须可分配；cost 只对 l<r 调用、无副作用；所有 f、cost 及候选和须严格在 (-inf,inf) 内。时间 O(n^2) 次 cost 与转移，空间 O(n^2)（f 为 long long，opt 为 int）。

```
template<class Cost>
vector<vector<long long>> knuth_interval_dp(int n, Cost cost,
                                            vector<vector<int>> &opt)
{
    using ll = long long;
    constexpr ll inf = LLONG_MAX / 4;
    assert(n >= 1);
    vector<vector<ll>> f(n + 1, vector<ll>(n + 1, inf));
    opt.assign(n + 1, vector<int>(n + 1, -1));
    for (int i = 1; i <= n; i++)
    {
        f[i][i] = 0;
        opt[i][i] = i;
    }
    for (int len = 2; len <= n; len++)
        for (int l = 1, r = len; r <= n; l++, r++)
        {
            ll w = cost(l, r);
            int hi = min(opt[l + 1][r], r - 1);
            for (int k = opt[l][r - 1]; k <= hi; k++)
            {
                ll value = f[l][k] + f[k + 1][r] + w;
                if (value < f[l][r])
                {
                    f[l][r] = value;
                    opt[l][r] = k;
                }
            }
        }
    return f;
}
```

模板题使用：AtCoder AWC0100 O Joining of Circular Tablets（环形石板合并）

题意：2≤N≤3000 块石板成环，重量 1≤A_i≤10^9；每次合并相邻两块，代价为新块重量，求合成一块的最小总代价。复制成长 2N-1 的链，f(l,r)=w(l,r)+min f(l,k)+f(k+1,r)，w 为区间和：四边形不等式取等、区间越大和越大，满足 Knuth 条件。答案取 min f(i,i+N-1)；O(N^2) 时间，表约 432MB（限 1024MB），答案 <10^16。

所需模板：knuth_interval_dp（第 29 页，4.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://atcoder.jp/contests/awc0100/tasks/awc0100_o

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n), sum(2 * n);
    for (auto &x : a) cin >> x;
    // Break the circle: positions 1..2n-1 repeat a, every window of n is a line.
    for (int i = 1; i < 2 * n; i++) sum[i] = sum[i - 1] + a[(i - 1) % n];
    auto cost = [&](int l, int r) { return sum[r] - sum[l - 1]; };
    vector<vector<int>> opt;
    auto f = knuth_interval_dp(2 * n - 1, cost, opt);
    long long answer = f[1][n];
    for (int i = 2; i <= n; i++) answer = min(answer, f[i][i + n - 1]);
    cout << answer << '\n';
}
```

4.3 WQS 二分

4.3.1 WQS 二分：至多 k 个不相邻位置

问题描述：给定数组 a 和 k，选出至多 k 个两两不相邻的位置，使所选元素之和最大。

核心思路：WQS 二分：恰选 j 个时的最优值关于 j 是凹的，于是给每个选中位置扣罚 λ，去掉个数限制做线性 DP：`solve` 中 `cur`、`prev` 分别是前 i 个、前 $i-1$ 个位置的最优（权和，个数），权和相同时取个数多者。罚 0 时最优个数已不超过 k 就直接返回；否则二分最大的非负整数 λ，使最优个数仍至少为 k，此时恰选 k 个也是该罚值下的最优解之一，答案为惩罚后最优值加 λk。

接口契约：wqs_independent_set(a,k) 返回 0-based 数组中至多选 k 个两两不相邻位置的最大权和；允许空选、空数组、负权，k>=0。n=a.size()<INT_MAX，W=max|a[i]|，要求 n*W<=LLONG_MAX/4（以足够宽类型核验），因此不含 LLONG_MIN。只返回最优值，不返回方案。路径独立集等价于路径边匹配，定基数最优值离散凹；每次选择扣 penalty，pair 比较先权和、并列取更多位置。若罚 0 的最优数量 <=k 直接返回；否则二分仍能选至少 k 个的最大非负整数罚值，答案为惩罚后最优值 +penalty*k。不是任意 DP 的 WQS 通用封装。O(n log(W+2)) 时间、O(1) 额外空间。

```
long long wqs_independent_set(const vector<long long> &a, int k)
{
    using ll = long long;
    assert(k >= 0);
    if (k == 0 || a.empty()) return 0;
    auto solve = [&](ll penalty) -> pair<ll, int>
    {
        pair<ll, int> prev{0, 0}, cur{0, 0};
        for (ll w : a)
        {
            pair<ll, int> take{prev.first + w - penalty, prev.second + 1};
            prev = cur;
            cur = max(cur, take);
        }
        return cur;
    };
    auto free = solve(0);
    if (free.second <= k) return free.first;
    ll lo = 0, hi = max(0LL, *max_element(a.begin(), a.end()));
    while (lo < hi)
    {
        ll mid = lo + (hi - lo + 1) / 2;
        if (solve(mid).second >= k) lo = mid;
        else hi = mid - 1;
    }
    return solve(lo).first + lo * k;
}
```

第五章　离线算法

5.1　CDQ 分治

5.1.1　静态三维偏序计数

问题描述：给定 n 个三维点，对每个点求有多少**其他**点的三个坐标都不大于它的对应坐标（三维偏序计数；坐标完全相同的点互相计入）。

核心思路：CDQ 分治。先按三维字典序排序，把完全相同的点合并成一组并记重数 `w`，第三维离散化。对组序列分治：左半的组在字典序上都排在右半之前，第一维已满足。两半递归返回后各自已按第二维 `y` 有序，归并时对右半每个组，把左半中 `y` 不超过它的组按第三维以重数加入树状数组，再查第三维前缀和累加到 `cnt`；归并完撤销左半贡献，同时完成按 `y` 的归并排序。最后每个点的答案为其组的 `cnt` 加同组其他点数 $w-1$。

代码依赖：Fenwick（第 69 页，C.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：count(points) 返回每个原输入位置被其他点逐坐标 ≤ 的次数，重复坐标仍是不同点，排除自身。输入 array<long long,3>，完整 signed64 坐标，空输入返回空；输入不变。内部按三维字典序合并重合点，压缩第三维，CDQ 归并与 Fenwick 回滚。O(n $\log^2$(n+1)) 时间、O(n) 空间、O(log(n+1)) 递归；n 须 ≤INT_MAX/2 且内存可分配，保证共享 Fenwick 的 int 步进不溢出。仅静态无权弱偏序，不含严格不等号、动态更新或 CDQ 递推。

```
struct Dominance3D
{
    static vector<int> count(const vector<array<long long, 3>> &p)
    {
        assert(p.size() <= INT_MAX / 2);
        int n = int(p.size());
        vector<int> order(n), group(n), w;
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int i, int j) { return p[i] < p[j]; });
        vector<array<long long, 3>> v;
        for (int i : order)
        {
            if (v.empty() || v.back() != p[i])
            {
                v.push_back(p[i]);
                w.push_back(0);
            }
            ++w.back();
            group[i] = int(v.size()) - 1;
        }
        int m = int(v.size());
        vector<long long> z;
        for (auto q : v) z.push_back(q[2]);
        sort(z.begin(), z.end());
        z.erase(unique(z.begin(), z.end()), z.end());
        struct Node
        {
            long long y;
            int z, w, id;
        };
        vector<Node> a(m), tmp(m);
        vector<int> cnt(m), ans(n);
        for (int i = 0; i < m; ++i)
            a[i] = {v[i][1], int(lower_bound(z.begin(), z.end(), v[i][2]) -
                                 z.begin()) + 1, w[i], i};
        Fenwick<int> bit(int(z.size()));
```

三维偏序的 CDQ 归并与原序返回（接上页同一 count 函数）：

```
        auto cdq = [&](auto &&self, int l, int r) -> void
        {
            if (r - l <= 1) return;
            int mid = l + (r - l) / 2;
            self(self, l, mid);
            self(self, mid, r);
            int i = l, k = l;
            for (int j = mid; j < r; ++j)
            {
                while (i < mid && a[i].y <= a[j].y)
                {
                    bit.add(a[i].z, a[i].w);
                    tmp[k++] = a[i++];
                }
                cnt[a[j].id] += bit.sum(a[j].z);
                tmp[k++] = a[j];
            }
            for (int t = l; t < i; ++t) bit.add(a[t].z, -a[t].w);
            while (i < mid) tmp[k++] = a[i++];
            copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
        };
        cdq(cdq, 0, m);
        for (int i = 0; i < n; ++i)
            ans[i] = cnt[group[i]] + w[group[i]] - 1;
        return ans;
    }
};
```

模板题使用：Luogu P3810 【模板】三维偏序 / 陌上花开

题意：1≤n≤100000，1≤ 三个坐标 ≤k≤200000。对每个输入点统计其他点三个坐标均 ≤ 自身的次数，输出次数为 0..n−1 的点数直方图。重复位置分别计数且各自排除自己。k 只约束题面坐标，库内部压缩，无需外部初始化；空输入和 signed64 极值另属 API 扩展。

所需模板：Fenwick（第 69 页，C.1 节）、Dominance3D（第 32 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3810

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<array<long long, 3>> p(n);
    for (auto &[x, y, z] : p) cin >> x >> y >> z;
    vector<int> hist(n);
    for (int x : Dominance3D::count(p)) ++hist[x];
    for (int x : hist) cout << x << '\n';
}
```

5.1.2 CDQ 卷积递推：贡献只跨越一次

`cdq_convolution` 求

$$f_i = b_i + \sum_{j<i} f_j g_{i-j}, \qquad g_0 = 0.$$

半开区间 $[l, r)$：先递归左半 $[l, m)$，再与 $g_{0\ldots r-l-1}$ 卷积加入右半，最后递归右半；顺序不可改。根层卷积长度 $n + \lfloor n/2 \rfloor - 1$ 须不超过 NTT 最大二次幂容量。时间 $O(n \log^2(n+1))$，额外空间 $O(n)$。

5.2 整体二分

5.2.1 整体二分：动态区间第 k 小

问题描述：数组支持单点赋值，并多次询问某一时刻闭区间 $[l, r]$ 中第 k 小的值；所有操作先登记，再离线一次求出全部答案。

核心思路：整体二分。初值和修改都拆成按时间排列的事件：在位置 `l` 加入某值（`k` 为 $+1$）或删去旧值（`k` 为 -1），询问是另一类事件；值先离散化。`solve` 处理答案位于值域 $[lo, hi]$ 的所有事件：按时间顺序扫描，值不超过 mid 的修改在树状数组的对应位置加上 `k`，询问查区间内这类值的个数 `smaller`；k 不超过它则答案在左半，否则 k 减去它进入右半，修改按值分到两半。扫完撤销树状数组，稳定划分保持时间顺序，再分别递归；值域缩成一点时即为该组询问的答案。

代码依赖：Fenwick（第 69 页，C.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：OverallKth(a) 读取 0-based 非空初始数组；后续位置用 1..n。set(pos,value) 登记单点赋值；query(l,r,k) 登记闭区间第 k 小并返回 0-based 查询 ID，1<=k<=r-l+1。run() 按查询 ID 返回 vector<long long>，答案对应登记查询时的状态，不是最后数组；可重复 run，也可继续登记后重新 run。支持完整 long long 值域，不做值的算术；同值修改跳过。依赖 Fenwick<int>，递归值域分治，保持时间顺序并撤销贡献。n<=INT_MAX/2，事件数 E=n+2U+Q<=INT_MAX（U 为实际变值次数），另须满足内存限制。V 为出现过的不同值数，时间 O(E log(E+1)+E log(n+1) log(V+1))，空间 O(n+E)。只读 current/events/queries，不外部修改。

```
struct OverallKth
{
    using ll = long long;

    struct Event
    {
        int l, r, k, id;
        ll value;
    };

    int n, queries = 0;
    vector<ll> current;
    vector<Event> events;

    OverallKth(const vector<ll> &a) : n(a.size()), current(a)
    {
        assert(n > 0);
        for (int i = 1; i <= n; i++)
        {
            events.push_back({i, 0, 1, -1, a[i - 1]});
        }
    }

    void set(int pos, ll value)
    {
        assert(1 <= pos && pos <= n);
        if (current[pos - 1] == value) return;
        events.push_back({pos, 0, -1, -1, current[pos - 1]});
        events.push_back({pos, 0, 1, -1, value});
        current[pos - 1] = value;
    }

    int query(int l, int r, int k)
    {
        assert(1 <= l && l <= r && r <= n);
        assert(1 <= k && k <= r - l + 1);
        int id = queries++;
        events.push_back({l, r, k, id, 0});
        return id;
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run() const
    {
        vector<ll> answer(queries), values;
        if (queries == 0) return answer;
        auto a = events;
        for (auto e : a)
        {
            if (e.id < 0) values.push_back(e.value);
        }
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        for (auto &e : a)
        {
            if (e.id < 0)
            {
                e.value = lower_bound(values.begin(), values.end(), e.value) - values.begin();
            }
        }
        vector<Event> buffer(a.size());
        vector<bool> left(a.size());
        Fenwick<int> bit(n);
```

继续求解（接上页同一结构体）：

```
        auto solve = [&](auto &&self, int lo, int hi, int begin, int end) -> void
        {
            if (begin == end) return;
            if (lo == hi)
            {
                for (int i = begin; i < end; i++)
                {
                    if (a[i].id >= 0) answer[a[i].id] = values[lo];
                }
                return;
            }
            int mid = lo + (hi - lo) / 2, count = 0;
            for (int i = begin; i < end; i++)
            {
                auto &e = a[i];
                if (e.id < 0)
                {
                    left[i] = e.value <= mid;
                    if (left[i]) bit.add(e.l, e.k);
                }
                else
                {
                    int smaller = bit.query(e.l, e.r);
                    left[i] = e.k <= smaller;
                    if (!left[i]) e.k -= smaller;
                }
                count += left[i];
            }
            int p = begin, q = begin + count;
            for (int i = begin; i < end; i++)
            {
                auto e = a[i];
                if (e.id < 0 && left[i]) bit.add(e.l, -e.k);
                if (left[i]) buffer[p++] = e;
                else buffer[q++] = e;
            }
            copy(buffer.begin() + begin, buffer.begin() + end, a.begin() + begin);
            self(self, lo, mid, begin, begin + count);
            self(self, mid + 1, hi, begin + count, end);
        };
        solve(solve, 0, int(values.size()) - 1, 0, a.size());
        return answer;
    }
};
```

5.3 莫队算法

5.3.1 普通莫队算法

5.3.1.1 曼哈顿 MST 的递归 DFS 查询顺序

功能：给定平面整数点（例如把莫队询问 $[l,r)$ 看作点 (l,r)），返回一个访问顺序，使相邻两点的曼哈顿距离之和不超过曼哈顿最小生成树权值的两倍，可作为莫队处理询问的顺序。

核心思路：先用 `ManhattanMST::solve` 求曼哈顿最小生成树并建邻接表，再从 0 号点递归 DFS，按先序输出。DFS 沿每条树边恰好往返一次，由三角不等式，先序中相邻两点的距离不超过它们在树上路径的长度，总和至多为树权的两倍。

代码依赖：ManhattanMST（第 85 页，G.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：输入二维整数点，返回从原编号 0 开始的 DFS 先序排列；空输入返回空序列，重复点分别保留。可把静态区间 [l,r) 看作点 (l,r)，按返回编号移动莫队双指针。只负责排序，统计和答案由调用处维护。

复用 ManhattanMST 构造树，递归 DFS，每个点恰好一次。设 MST 权值为 T，由 DFS 往返及三角不等式，相邻访问点的 L1 距离之和 ≤2T。起始空区间到首个查询的移动另计；排序 O(q log(q+1)) 时间、O(q) 空间与最坏 O(q) 递归栈。不保证比普通分块更快，也不保证最短访问顺序。点坐标可为 signed64，距离界用 int128。查询数须 ≤INT_MAX；输入不变。P1494 当前实现已 100 分 AC，10 点单点最大 113ms、峰值 10.49MB；排名未核查。

```
vector<int> manhattan_order(const vector<ManhattanMST::Point> &p)
{
    assert(p.size() <= INT_MAX);
    int n = int(p.size());
    if (n == 0)
        return {};
    auto t = ManhattanMST::solve(p);
    vector<vector<int>> g(n);
    for (auto [u, v] : t.edges)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> ord;
    ord.reserve(n);
    auto dfs = [&](auto &&self, int u, int fa) -> void
    {
        ord.push_back(u);
        for (int v : g[u])
        {
            if (v != fa)
                self(self, v, u);
        }
    };
    dfs(dfs, 0, -1);
    return ord;
}
```

5.3.1.2 普通莫队：静态区间统计

功能：普通莫队调度器：离线回答静态数组上的区间询问 $[l,r)$，调用方提供加入位置、删除位置、记录答案三个回调，适用于能快速增删一个元素的统计。

核心思路：按左端点分块，块号小者在前；同块内右端点按块号奇偶交替升序、降序（蛇形），避免右指针每换一块都从头回跳。依次处理询问时，当前区间 $[l,r)$ 先向外扩张两端再向内收缩，保证删除的位置一定仍在区间

内。全部结束后删除剩余元素，使统计回到空状态。

```
struct Mo
{
    struct Query
    {
        int l, r, id;
    };
    int n;
    vector<Query> q;

    Mo(int n) : n(n)
    {
        assert(n >= 0);
    }
    int add(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        int id = q.size();
        q.push_back({l, r, id});
        return id;
    }
    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block;
            int y = b.l / block;
            if (x != y) return x < y;
            return x & 1 ? a.r > b.r : a.r < b.r;
        });
        int l = 0;
        int r = 0;
        for (auto a : ord)
        {
            while (l > a.l) add(--l);
            while (r < a.r) add(r++);
            while (l < a.l) del(l++);
            while (r > a.r) del(--r);
            ans(a.id);
        }
        while (l < r) del(l++);
    }
};
```

接口契约： Mo(n)，n≥0；add(l,r) 登记 0-based 半开区间并返回查询编号，允许空区间。run(add,del,ans,block=0) 中加删回调接收下标，答案回调接收原查询编号。开始时统计状态须为空，结束后删除剩余点恢复空状态；可重复 run，运行期间不能修改查询表。适用于加删任意位置语义相同的集合统计；逆序对等顺序敏感统计需另外区分左右端点。

查询按左端块号、右端蛇形排序；先扩张再收缩，保证删除位置仍在当前区间。回调 O(1) 时，正块长 B 的时间 O(q log(q+1)+qB+n^2/B+n)，额外空间 O(q)。默认 B=max(1,floor(n/sqrt(q)))，q=0 直接返回。n 和 q 须能用 int 表示。P2709 频次平方和须用 long long，最大 n^2=10^10。P2709 当前代码已 AC（30 个测试点，单点最大 69ms）；运行时间排名未核查。曼哈顿最小生成树排序：把查询 [l,r) 看作点 (l,r)，用 ManhattanMST 建树后按 DFS 先序处理查询，相邻查询的指针移动总和不超过 2 倍 MST 权；不保证比分块排序更快，一般题直接用默认分块。

模板题使用：P2709 莫队 / 小 B 的询问

题意：静态数组值域 1..k，询问闭区间内各值出现次数的平方和。n,m,k≤100000，输入 1-based 闭区间转成 0-based 半开区间；加删一个值由平方差 O(1) 更新，答案用 long long。

所需模板：Mo（第 39 页，5.3.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2709

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    Mo mo(n);
    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        mo.add(l - 1, r);
    }
    vector<int> cnt(k + 1);
    vector<long long> ans(m);
    long long sum = 0;
    auto add = [&](int i)
    {
        sum += 2LL * cnt[a[i]] + 1;
        cnt[a[i]]++;
    };
    auto del = [&](int i)
    {
        cnt[a[i]]--;
        sum -= 2LL * cnt[a[i]] + 1;
    };
    mo.run(add, del, [&](int id)
    {
        ans[id] = sum;
    });
    for (long long x : ans) cout << x << '\n';
}
```

5.3.2 带修改莫队

5.3.2.1 带修改的莫队

功能：带修改莫队的调度器：离线登记“前 t 次修改完成后对区间 $[l,r)$ 的询问”，由调用方提供加点、删点、执行或撤销修改、记录答案四个回调，按优化顺序移动指针回答全部询问。

核心思路：把询问看作三维点 (l,r,t)：先按 l 所在块、再按 r 所在块、最后按 t 排序，第二、三关键字按奇偶交替升降（蛇形），减少指针来回跳动；块长默认取 $n^{2/3}$ 量级。处理每个询问时先移动时间指针：前进时执行第 t 次修改、后退时撤销，回调同时拿到当前区间，以便修正区间内的贡献；再先扩张后收缩地移动左右端点，最后调用 `ans`。

接口契约：ModifiedMo(n,m) 保存离线询问，n>0，m 为修改总数。add(l,r,t) 登记第 t 次修改完成后的 [l,r)，返回从 0 开始的询问编号；0≤l≤r≤n，0≤t≤m，询问数小于 INT_MAX。run(add,del,chg,ans,block=0) 排序并移动三个指针。add(p)/del(p) 按下标更新当前区间统计；chg(i,forward,l,r) 将编号 i 的修改前进或回退，并同步修正当前 [l,r) 内的贡献；ans(id) 记录结果。修改编号从 0 开始，前进第 i 次修改时原时间为 i，回退时原时间为 i+1。回调初始状态必须为原数组、空区间、时间 0；重复 run 前由调用方重置状态。状态不会自动恢复，原询问顺序不变。仅适用于可增删维护的区间统计，需区分左右端操作时另行拆分回调。默认块长约 n^(2/3)，也可显式指定正块长。n、询问数、修改数同阶且回调 O(1) 时总时间 O(n^(5/3))，调度器空间 O(q)。

```
struct ModifiedMo
{
    struct Query
    {
        int l, r, t, id;
    };

    int n, m;
    vector<Query> q;

    ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }

    int add(int l, int r, int t)
    {
        assert(0 <= l && l <= r && r <= n);
        assert(0 <= t && t <= m);
        int id = q.size();
        q.push_back({l, r, t, id});
        return id;
    }
```

```
    template <class Add, class Del, class Chg, class Ans>
    void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
        assert(block > 0);
        auto ord = q;
        auto cmp = [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            int p = a.r / block, q = b.r / block;
            if (p != q) return (x & 1) ? p > q : p < q;
            return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
        };
        sort(ord.begin(), ord.end(), cmp);
        int l = 0, r = 0, t = 0;
        for (auto q : ord)
        {
            while (t < q.t)
            {
                chg(t, true, l, r);
                t++;
            }
            while (t > q.t)
            {
                t--;
                chg(t, false, l, r);
            }
            while (l > q.l) add(--l);
            while (r < q.r) add(r++);
            while (l < q.l) del(l++);
            while (r > q.r) del(--r);
            ans(q.id);
        }
    }
};
```

模板题使用：Luogu P1903

题意：单点改色、区间不同颜色数，n,m≤133333，颜色 1..1000000。先读完操作，cur 仅用于记录每次修改前后的颜色，a 保持初始数组。题目闭区间 [L,R] 转为 [L-1,R)，询问时间是此前修改数。chg 必须先删旧贡献、改 a[p]、再加新贡献，回退使用 pre。按登记顺序输出。三维莫队，回调 O(1)，同阶规模时 O(n^(5/3))。

所需模板：ModifiedMo（第 42 页，5.3.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1903

Luogu P1903：使用代码

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<int> a(n);
    for (int &v : a) scanf("%d", &v);
    vector<int> cur = a;
    vector<array<int, 3>> upd;
    vector<array<int, 3>> qry;
    for (int i = 0; i < m; i++)
    {
        char op;
        int x, y;
        scanf(" %c%d%d", &op, &x, &y);
        x--;
        if (op == 'Q')
            qry.push_back({x, y, int(upd.size())});
        else
        {
            upd.push_back({x, cur[x], y});
            cur[x] = y;
        }
    }
    ModifiedMo mo(n, upd.size());
    for (auto [l, r, t] : qry) mo.add(l, r, t);
    vector<int> cnt(1000001), res(qry.size());
    int tot = 0;
    auto add = [&](int p)
    {
        if (cnt[a[p]]++ == 0) tot++;
    };
    auto del = [&](int p)
    {
        if (--cnt[a[p]] == 0) tot--;
    };
    auto chg = [&](int t, bool fwd, int l, int r)
    {
        auto [p, pre, nxt] = upd[t];
        bool in = l <= p && p < r;
        if (in) del(p);
        a[p] = fwd ? nxt : pre;
        if (in) add(p);
    };
    auto ans = [&](int id)
    {
        res[id] = tot;
    };
    mo.run(add, del, chg, ans);
    for (int v : res) printf("%d\n", v);
}
```

5.3.2.2 带修莫队：时间轴与区间状态

离线记录每个单点修改的下标、修改前值和修改后值；同一位置的旧值取上次修改后的数组。离散化须纳入全部初值与修改后值。询问时间 t 表示前 t 次修改已执行。维护 (l, r, t) 与半开区间 $[l, r)$；前进写 after、回退写 before，修改点在当前区间内时先删旧贡献、改值、再加新贡献。run 结束后状态停在最后一条询问，再次调用须重置。左块、右块、时间三关键字排序（蛇形），时间为真实修改次数。块长 B、询问 Q、修改 U 时移动量

$$O\left(QB + \frac{n^2}{B} + U\min\left(Q, (\lfloor n/B \rfloor + 1)^2\right)\right),$$

另加 $O(Q \log Q)$ 排序、$O(Q)$ 空间；同阶时 $B \approx n^{2/3}$，总 $O(n^{5/3})$。需区分左右端增删时应拆分回调。

5.3.3 树上莫队

5.3.3.1 树上莫队：点权与边权

功能： 树上莫队调度器：离线回答若干条树上路径 u–v 的统计询问（点权；或把边权记在子节点上的边权），调用方提供加点、删点、记录答案三个回调。

核心思路： DFS 生成括号序 `euler`（每个点进出各记一次，长 $2n$）。设 $\text{in}_u \le \text{in}_v$：若 u 是 v 的祖先，路径上的点恰是区间 $[\text{in}_u, \text{in}_v]$ 中出现奇数次的点；否则取 $[\text{out}_u, \text{in}_v]$，其中缺 LCA，记为 `extra`。边权询问正好相反：祖先情形要从区间中去掉 LCA，记为 `extra`，另一情形无需补。于是化为括号序上的普通莫队，移动端点时用 `toggle` 翻转点的在/不在状态，出现两次的点自动抵消；回答前后各翻转一次 `extra`。

代码依赖：HLD（第 76 页，E.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： 先对非空连通树 HLD.build(root)，再 TreeMo(tree)；tree 须活到 run 结束且不能修改。点号 1..n，双欧拉序长 2n，n<=INT_MAX/2；DFS 递归。add(u,v,edge=false) 返回 0-based 查询 ID；默认点权，edge=true 把父边权记在子点并排除 LCA。run(add,del,ans,block=0) 的 add/del 参数是顶点，ans 参数是查询 ID；只适合与集合排列无关且可增删的统计。调用前统计为空，结束恢复空；不可在回调内修改树/查询，可重复运行。混用点边查询时回调须有一致的数据解释。默认块长 max(1,2n/sqrt(m))；自定 B>0。m>0 时调度 O(m log(m+1)+mB+n^2/B+n)，查询转换另需 O(m log n)，回调代价另计。空间 O(n+m)，复用 HLD；整数大小另须满足内存限制。

```
struct TreeMo
{
    struct Query
    {
        int l, r, extra, id;
    };

    const HLD &tree;
    vector<int> in, out, euler;
    vector<Query> q;

    TreeMo(const HLD &tree) : tree(tree), in(tree.n + 1), out(tree.n + 1)
    {
        assert(tree.n > 0 && tree.timer == tree.n);
        dfs(tree.rk[1], 0);
    }

    void dfs(int u, int p)
    {
        in[u] = euler.size();
        euler.push_back(u);
        for (int v : tree.g[u])
        {
            if (v == p) continue;
            dfs(v, u);
```

```
        }
        out[u] = euler.size();
        euler.push_back(u);
    }

    int add(int u, int v, bool edge = false)
    {
        assert(1 <= u && u <= tree.n && 1 <= v && v <= tree.n);
        if (in[u] > in[v]) swap(u, v);
        int w = tree.lca(u, v);
        int l = w == u ? in[u] : out[u];
        int extra = w == u ? 0 : w;
        if (edge) extra = w == u ? w : 0;
        int id = q.size();
        q.push_back({l, in[v] + 1, extra, id});
        return id;
    }
```

查询调度与状态恢复（接上页同一结构体）：

```
    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        int n = euler.size();
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return (x & 1) ? a.r > b.r : a.r < b.r;
        });
        vector<bool> active(tree.n + 1);
        auto toggle = [&](int u)
        {
            if (active[u]) del(u);
            else add(u);
            active[u] = !active[u];
        };
        int l = 0, r = 0;
        for (auto a : ord)
        {
            while (l > a.l) toggle(euler[--l]);
            while (r < a.r) toggle(euler[r++]);
            while (l < a.l) toggle(euler[l++]);
            while (r > a.r) toggle(euler[--r]);
            if (a.extra) toggle(a.extra);
            ans(a.id);
            if (a.extra) toggle(a.extra);
        }
        for (int u = 1; u <= tree.n; u++)
        {
            if (active[u]) del(u);
        }
    }
};
```

5.3.4 回滚莫队

5.3.4.1 回滚莫队

功能：回滚莫队调度器：离线回答区间询问，统计信息只需支持插入和整体回滚到快照，不需要删除（例如区间内相同值的最远距离）。

核心思路：询问按左端点所在块分组，组内按右端点升序。对一块，令 `end` 为块的右边界，右指针 `r` 从 `end` 起只向右扩展，维护 $[\text{end}, r)$ 的统计。处理询问时先把 `r` 推到询问右端点，拍快照，再从 `end` 向左逐个插入到询问左端点，记录答案后回滚到快照，左侧插入因此不会累积。右端点不超出本块的短询问直接暴力插入再回滚；换块时回滚到最初的空快照。

思路中的情况	代码中的处理	行
进入新块	回滚到空状态，右指针重置为块右边界 `end`	43
询问在块内	拍快照，暴力插入 $[l, r)$，回答后回滚	49
右侧扩展	右指针单调右移到询问右端点，这部分保留	57
左侧临时插入	拍快照后从 `end` 向左插入到 l，回答后回滚	59

接口契约： RollbackMo(n)，n>=0；add(l,r) 登记 0-based 半开区间 [l,r)，允许空区间，返回 0-based 查询 ID。run(add, snapshot, rollback, ans, block=0)：add 接收位置，snapshot 返回当前快照，rollback 恢复该祖先快照，ans 接收查询 ID。开始时统计必须为空；结束恢复初始快照。每次 add 都须可撤销，包含长期右段；不保留失效快照、不修改查询。只支持与插入次序无关的区间统计。默认 B=max(1,n/sqrt(m))；m>0 时调度与 O(1) 增删总计 O(m log(m+1)+mB+n^2/B+n)，空间 O(m) 加回调状态/历史 O(n)。可重复 run，n=0 只允许空查询；无查询不调用回调。

```
struct RollbackMo
{
    struct Query
    {
        int l, r, id;
    };

    int n;
    vector<Query> q;

    RollbackMo(int n) : n(n)
    {
        assert(n >= 0);
    }

    int add(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        int id = q.size();
        q.push_back({l, r, id});
        return id;
    }

    template <class Add, class Snap, class Undo, class Ans>
    void run(Add add, Snap snapshot, Undo rollback, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return a.r < b.r;
        });
```

查询调度与状态恢复（接上页同一结构体）：

```
        auto empty = snapshot();
        int last = -1, r = 0;
        for (auto a : ord)
        {
            int b = a.l / block;
            int end = a.l + min(block - a.l % block, n - a.l);
            if (b != last)
            {
                rollback(empty);
                last = b;
                r = end;
            }
            if (a.r <= end)
            {
                auto saved = snapshot();
                for (int i = a.l; i < a.r; i++) add(i);
                ans(a.id);
                rollback(saved);
                continue;
            }
            while (r < a.r) add(r++);
            auto saved = snapshot();
            for (int i = end; i > a.l; ) add(--i);
            ans(a.id);
            rollback(saved);
        }
        rollback(empty);
    }
};

struct TreeMo
{
    struct Query
    {
        int l, r, extra, id;
    };

    const HLD &tree;
    vector<int> in, out, euler;
    vector<Query> q;

    TreeMo(const HLD &tree) : tree(tree), in(tree.n + 1), out(tree.n + 1)
    {
        assert(tree.n > 0 && tree.timer == tree.n);
        dfs(tree.rk[1], 0);
    }

    void dfs(int u, int p)
    {
        in[u] = euler.size();
        euler.push_back(u);
        for (int v : tree.g[u])
        {
            if (v == p) continue;
```

```
            dfs(v, u);
        }
        out[u] = euler.size();
        euler.push_back(u);
    }

    int add(int u, int v, bool edge = false)
    {
        assert(1 <= u && u <= tree.n && 1 <= v && v <= tree.n);
        if (in[u] > in[v]) swap(u, v);
        int w = tree.lca(u, v);
        int l = w == u ? in[u] : out[u];
        int extra = w == u ? 0 : w;
        if (edge) extra = w == u ? w : 0;
        int id = q.size();
        q.push_back({l, in[v] + 1, extra, id});
        return id;
    }

    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        int n = euler.size();
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return (x & 1) ? a.r > b.r : a.r < b.r;
        });
        vector<bool> active(tree.n + 1);
        auto toggle = [&](int u)
        {
            if (active[u]) del(u);
            else add(u);
            active[u] = !active[u];
        };
        int l = 0, r = 0;
        for (auto a : ord)
        {
            while (l > a.l) toggle(euler[--l]);
            while (r < a.r) toggle(euler[r++]);
            while (l < a.l) toggle(euler[l++]);
            while (r > a.r) toggle(euler[--r]);
            if (a.extra) toggle(a.extra);
            ans(a.id);
            if (a.extra) toggle(a.extra);
        }
        for (int u = 1; u <= tree.n; u++)
        {
            if (active[u]) del(u);
        }
    }
```

```
};
```

模板题使用：P5906 【模板】回滚莫队 & 不删除莫队

题意：n,m<=200000，值 1..2e9；对每个 1-based 闭区间输出相同值的最远下标差，无重复输出 0。离散化，保存各值最左/最右位置和全局答案，历史逐次恢复。

所需模板：RollbackMo（第 47 页，5.3.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5906

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n), values;
    for (int &x : a) cin >> x;
    values = a;
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    for (int &x : a)
    {
        x = lower_bound(values.begin(), values.end(), x) - values.begin();
    }
    int m;
    cin >> m;
    RollbackMo mo(n);
    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        mo.add(l - 1, r);
    }
```

继续处理操作（接上页同一 main）：

```
    struct Change
    {
        int color, first, last, best;
    };
    vector<Change> history;
    vector<int> first(values.size(), n), last(values.size(), -1), answer(m);
    int best = 0;
    auto add = [&](int i)
    {
        int c = a[i];
        history.push_back({c, first[c], last[c], best});
        first[c] = min(first[c], i);
        last[c] = max(last[c], i);
        best = max(best, last[c] - first[c]);
    };
    auto snapshot = [&]()
    {
        return history.size();
    };
    auto rollback = [&](size_t saved)
    {
        while (history.size() > saved)
        {
            auto h = history.back();
            history.pop_back();
            first[h.color] = h.first;
            last[h.color] = h.last;
            best = h.best;
        }
    };
    mo.run(add, snapshot, rollback, [&](int id)
    {
        answer[id] = best;
    });
    for (int x : answer) cout << x << '\n';
}
```

5.3.5 莫队二次离线

5.3.5.1 莫队二次离线：异或位数配对

问题描述：给定数组 a 和整数 k，多次询问区间 $[l,r)$ 内有多少对下标 $i<j$ 满足 $a_i \oplus a_j$ 的二进制恰有 k 个 1。

核心思路：莫队二次离线。设 $c(i,P)$ 为前缀 $[0,P)$ 中与 a_i 配对成功的下标数。莫队每把端点移过一段下标，答案变化都是若干项 $\pm\big(c(i,i)-c(i,P)\big)$ 之和，P 为另一端点（左端移动且 $k=0$ 时还要扣掉自身配对）。$c(i,i)$ 与询问无关，用前缀和 `prefix` 直接加减；$c(i,P)$ 则作为事件“下标段对前缀 P”挂到 `events[P]`。第二遍从左到右扫描前缀，`count[v]` 为已扫前缀中与 v 配对的个数（每加入一个数，枚举所有含 k 个 1 的掩码更新），逐个回答事件。得到的是莫队顺序下相邻询问的答案差，最后按该顺序求前缀和。

思路中的情况	代码中的处理	行
预处理 $c(i,i)$	按下标扫描，前缀和 `prefix` 累计每个数与前面数的配对数	21
莫队排序	按左端点分块，块内右端点奇偶交替升降	24
右端点右移	加上 `prefix` 差，减去事件“新增段对前缀 `l`”	47
左端点左移	减去 `prefix` 差，加上事件“新增段对前缀 `r`”	53
第二次离线扫描	`count` 加入 a_{p-1} 后，回答挂在前缀 p 上的事件	77
扣除自身配对	$k=0$ 时 a_i 与自身也满足条件，位于前缀内则减一	80
还原答案	按莫队顺序对答案差做前缀和	87

接口契约： xor_hamming_pairs(a,queries,k,bits=14) 按查询输入顺序返回 long long 答案，统计 l≤i<j<r 且 popcount(a[i] xor a[j])=k 的下标对。queries 为 0-based 半开区间，允许空区间、重复查询与空数组；0≤bits≤20，0≤a[i]<2^bits，n、m 和容器下标须可由 int 表示。k 超出 [0,bits] 直接返回全零；相等值来自不同下标才构成一对。莫队移动生成前缀事件，第二次离线按前缀扫描；k=0 显式扣掉自身。令 C=binom(bits,k)，时间 O(2^bits+nC+m log(m+1)+n$\sqrt{\text{m}}$+m)，空间 O(2^bits+n+m)，m=0 直接返回。无需维护可逆的在线窗口，也不支持修改。

```cpp
// Queries are 0-based half-open; count unordered pairs of distinct indices.
vector<long long> xor_hamming_pairs(const vector<int> &a,
                                    const vector<pair<int, int>> &queries,
                                    int k,
                                    int bits = 14)
{
    int n = a.size(), m = queries.size();
    vector<long long> answer(m);
    assert(0 <= bits && bits <= 20);
    int universe = 1 << bits;
    for (int x : a) assert(0 <= x && x < universe);
    for (auto [l, r] : queries) assert(0 <= l && l <= r && r <= n);
    if (!m || k < 0 || k > bits) return answer;
    vector<int> masks;
    for (int x = 0; x < universe; x++)
        if (popcount(unsigned(x)) == k) masks.push_back(x);
    vector<int> count(universe);
    vector<long long> prefix(n + 1);
    for (int i = 0; i < n; i++)
    {
        prefix[i + 1] = prefix[i] + count[a[i]];
        for (int mask : masks) count[a[i] ^ mask]++;
    }
    int block = max(1, int(n / sqrt(m)));
    vector<int> order(m);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(),
         order.end(),
         [&](int x, int y)
         {
             int bx = queries[x].first / block, by = queries[y].first / block;
             if (bx != by) return bx < by;
             return bx & 1 ? queries[x].second > queries[y].second
                           : queries[x].second < queries[y].second;
         });

    struct Event
    {
        int id, l, r, sign;
    };

    vector<vector<Event>> events(n + 1);
```

莫队端点移动与前缀事件（接上页同一函数）：

```
    int l = 0, r = 0;
    for (int id : order)
    {
        auto [ql, qr] = queries[id];
        if (r < qr)
        {
            events[l].push_back({id, r, qr, -1});
            answer[id] += prefix[qr] - prefix[r];
            r = qr;
        }
        if (l > ql)
        {
            events[r].push_back({id, ql, l, 1});
            answer[id] -= prefix[l] - prefix[ql];
            l = ql;
        }
        if (r > qr)
        {
            events[l].push_back({id, qr, r, 1});
            answer[id] -= prefix[r] - prefix[qr];
            r = qr;
        }
        if (l < ql)
        {
            events[r].push_back({id, l, ql, -1});
            answer[id] += prefix[ql] - prefix[l];
            l = ql;
        }
    }
```

第二次离线与差分还原（接上页同一函数）：

```
    fill(count.begin(), count.end(), 0);
    for (int p = 0; p <= n; p++)
    {
        if (p)
            for (int mask : masks) count[a[p - 1] ^ mask]++;
        for (auto e : events[p])
            for (int i = e.l; i < e.r; i++)
            {
                int value = count[a[i]] - (k == 0 && i < p);
                answer[e.id] += 1LL * e.sign * value;
            }
    }
    long long total = 0;
    for (int id : order)
    {
        total += answer[id];
        answer[id] = total;
    }
    return answer;
}
```

模板题使用：Luogu P4887

题意：统计每个区间内异或值恰有 k 个 1 的无序下标对。输入 1-based 闭区间 [l,r]，只将 l 减一转为 0-based 半开区间；k 可以大于 14，此时全零。k=0 统计相等值的不同下标，不计自身。

所需模板：xor_hamming_pairs（第 52 页，5.3.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4887

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    vector<pair<int, int>> queries(m);
    for (auto &[l, r] : queries)
    {
        cin >> l >> r;
        l--;
    }
    for (long long x : xor_hamming_pairs(a, queries, k)) cout << x << '\n';
}
```

5.3.5.2 莫队二次离线：前缀事件与自身扣除

区间内 $H(i,j)=[\text{popcount}(a_i \text{xor} a_j)=k]$ 的无序对数，用 `xor_hamming_pairs`。$t(i)=\sum_{j<i} H(i,j)$，$T(p,i)=\sum_{j<p} H(i,j)-[k=0 \wedge i<p]$；右扩贡献 $t(i)-T(L,i)$，左扩 $T(R,i)-t(i)$，删除取反；t 用前缀和，其余按前缀长度存事件顺扫。0-based 半开区间，允许空区间；位宽默认 14，k 超位宽答案全零。$C=\binom{b}{k}$，时间 $O(2^b+nC+m\log(m+1)+n\sqrt{m}+m)$，空间 $O(2^b+n+m)$；答案用 long long。

第六章　分数规划

6.1　0/1 分数规划：Dinkelbach

问题描述： 0/1 分数规划：在所有可行方案中最大化 $\sum A/\sum B$（要求 $\sum B>0$）；方案的结构由调用方的回调 `best` 描述。

核心思路： 对给定 λ，$\max(A-\lambda B)>0$ 当且仅当最优比值大于 λ。Dinkelbach 迭代：用 `best` 求出使 $A-\lambda B$ 最大的方案，若该值已不超过（相对的）`eps`，当前 λ 就是最优比值；否则令 λ 为这个方案的比值 A/B，它严格大于旧的 λ，继续迭代。

接口契约： dinkelbach(best,lambda,eps=1e-10) 求可行方案中 $\sum$A/$\sum$B 的最大值，要求所有可行方案 $\sum$B>0。best(λ) 返回使 A-λB 最大的方案的 {A,B}（double）；初值 lambda 取任一可行方案的比值，不能随意取 0（所有比值为负时会出错）。每轮 $\lambda\leftarrow$A/B 单调上升，A-λB$\le$eps $\cdot$ max(1,|B|) 时返回。迭代次数一般远少于二分；最小化比值时把 a 取负或回调改为求最小并反转比较。原理、二分写法与常见组合见知识节。

```cpp
// Maximize (sum a)/(sum b) over feasible choices with sum b > 0.
// best(lambda) returns {A, B} of a choice maximizing A - lambda * B; lambda starts at the
// ratio of any feasible choice. Lambda increases every round and stops at the optimum.
template <class F> double dinkelbach(F best, double lambda, double eps = 1e-10)
{
    while (true)
    {
        auto [A, B] = best(lambda);
        if (A - lambda * B <= eps * max(1.0, fabs(B))) return lambda;
        lambda = A / B;
    }
}
```

6.2　0/1 分数规划

问题：在可行方案集合中最大化 $\dfrac{\sum a_i x_i}{\sum b_i x_i}$，$x_i\in\{0,1\}$，要求所有可行方案 $\sum b_i x_i>0$。

转化：令 $F(\lambda)=\max\left(\sum a_i x_i-\lambda\sum b_i x_i\right)$，它关于 λ 严格单调递减，最优比值 λ^* 恰为 $F(\lambda)=0$ 的根：$F(\lambda)\ge 0\iff\lambda\le\lambda^*$。求最小比值时把 max 换成 min、不等号反向。

二分：在答案范围内二分 λ，每次求一次 $F(\lambda)$ 判断正负；实数二分固定 60 到 100 轮，或按题目精度停止。总复杂度为轮数乘一次判定。

Dinkelbach：从任一可行方案的比值出发，取使 $F(\lambda)$ 达到最大的方案 (A,B)，令 $\lambda\leftarrow A/B$；λ 单调上升，$F(\lambda)\le$ 精度时即为最优，迭代次数通常远少于二分。模板见第 57 页，回调只需返回该方案的 (A,B)。

常见组合：

- 恰选 k 个：按 $a_i-\lambda b_i$ 排序取前 k 个，$O(n\log n)$ 判定。
- 最优比率生成树：边权取 $a_e-\lambda b_e$，求最大（或最小）生成树。
- 最优比率环（最小化环上 $\sum a/\sum b$）：边权取 $a_e-\lambda b_e$，存在负环 $\iff\lambda>$ 最优比值，用第 82 页 SPFA 判负环后二分；Dinkelbach 需要求出最负的环，通常直接二分。
- 带上界约束的选择（背包、树上连通块）：判定内部换成对应 DP，DP 的目标改成 $\sum(a-\lambda b)$。

注意：b_i 可正可负时单调性不成立，不能直接套用；判定必须求出“该 λ 下的最优方案”，不能用贪心近似。整数数据要精确答案时，可在最终方案上用分数 A/B 输出。

第七章　表达式求值

7.1　整数表达式求值

问题描述：给定含非负整数、加减乘除、取模、乘方、括号与一元正负号的表达式字符串，按通常的优先级和结合性计算其整数值。

核心思路：递归下降，每一级优先级对应一个函数：`expr` 处理加减、`term` 处理乘除模，都用循环实现左结合；`unary` 处理一元正负号；`power` 先读一个 `atom`（数字或括号内的 `expr`），遇到乘方号再以 `unary` 读指数，由此乘方右结合，且比一元负号结合更紧（$-2^2=-4$），指数用快速幂计算。`peek` 跳过空格看下一个字符，决定进入哪个分支。

接口契约：Expression(s).eval() 计算整数表达式 s 的值（long long）。支持非负整数字面量、+ - * / % ^、括号与一元正负号，空格忽略。优先级从高到低：^（右结合，指数须非负，0^0=1）、一元正负、* / %、+ -，同级左结合；/ 和% 按 C++ 截断，除数不能为 0。递归下降，O(|s|) 时间，递归深度等于括号与一元符号的嵌套深度。所有中间结果须在 long long 内；需要取模时把各运算改成模运算即可。非法输入触发 assert。

```
// Integer expression: + - * / % ^, parentheses, unary + -, spaces ignored.
// Precedence: ^ (right-assoc) > unary sign > * / % > + -; / and % truncate as in C++.
struct Expression
{
    using ll = long long;
    string s;
    size_t i = 0;

    explicit Expression(string text) : s(move(text)) {}

    char peek()
    {
        while (i < s.size() && s[i] == ' ') i++;
        return i < s.size() ? s[i] : 0;
    }

    ll atom()
    {
        if (peek() == '(')
        {
            i++;
            ll v = expr();
            assert(peek() == ')');
            i++;
            return v;
        }
        assert(isdigit(peek()));
        ll v = 0;
        while (i < s.size() && isdigit(s[i])) v = v * 10 + (s[i++] - '0');
        return v;
    }

    ll power()
    {
        ll a = atom();
        if (peek() != '^') return a;
```

```
        i++;
        ll e = unary(), r = 1; // right-assoc: a^b^c = a^(b^c)
        assert(e >= 0);
        while (e)
        {
            if (e & 1) r *= a;
            if (e >>= 1) a *= a; // no square after the last bit
        }
        return r;
    }

    ll unary()
    {
        char c = peek();
        if (c == '+' || c == '-')
        {
            i++;
            ll v = unary();
            return c == '-' ? -v : v;
        }
        return power();
    }

    ll term()
    {
        ll v = unary();
        for (char c; (c = peek()) == '*' || c == '/' || c == '%';)
        {
            i++;
            ll b = unary();
            assert(c == '*' || b != 0);
            v = c == '*' ? v * b : c == '/' ? v / b : v % b;
        }
        return v;
    }

    ll expr()
    {
        ll v = term();
        for (char c; (c = peek()) == '+' || c == '-';)
        {
            i++;
            ll b = term();
            v = c == '+' ? v + b : v - b;
        }
        return v;
    }
```

```
84
85      // Value of the whole string; the caller keeps every intermediate result within long long.
86      ll eval()
87      {
88          i = 0;
89          ll v = expr();
90          assert(peek() == 0);
91          return v;
92      }
93  };
```

第八章　珂朵莉树/颜色段均摊

8.1　珂朵莉树（颜色段均摊）

功能：珂朵莉树：维护分段常值的序列，支持把区间推平为同一值，以及按段遍历区间（在回调中修改段值、求和、求第 k 小等）。

核心思路：mp 以段起点为键、段值为值，键 hi 作哨兵结束最后一段。split(x) 找到包含 x 的段，若 x 不是段起点就插入一个以 x 为起点、值相同的新段，使 x 成为分界。assign 先切出 r 与 l，删除其间所有段后插入一个新段；each 切出区间后逐段调用回调。推平不断把多段合成一段，随机数据下段数保持很少。

接口契约：ODT<T>(lo,hi,v) 维护半开区间 [lo,hi) 上的分段常值序列，初值 v；mp 的键为段起点，键 hi 为哨兵。split(x) 使 x (lo≤x≤hi) 成为段起点并返回其迭代器；assign(l,r,v) 把 [l,r) 推平为 v (l≤r，空区间不变)；each(l,r,f) 依次对 [l,r) 内每段调用 f(段起点, 段终点, 值引用)，可在 f 中修改值、统计和、收集 (值, 长度) 求第 k 小或幂和。map 插入不使已有迭代器失效，故 split 的先后无关；经典 set 写法（删除旧段再插入两段）必须先 split(r) 再 split(l)。复杂度依赖推平操作：数据随机且推平占常数比例时段数期望 O(log n)，总复杂度约 O(m log n)；没有推平或被构造数据卡时会退化为 O(n) 每次。

```
// Chtholly tree on [lo, hi): map keys are segment starts, the key hi closes the last segment.
template <class T> struct ODT
{
    map<long long, T> mp;

    ODT(long long lo, long long hi, T v = T())
    {
        mp[lo] = v;
        mp[hi] = T();
    }

    // Make x a segment start (lo <= x <= hi) and return its iterator.
    typename map<long long, T>::iterator split(long long x)
    {
        auto it = prev(mp.upper_bound(x));
        if (it->first == x) return it;
        return mp.emplace_hint(next(it), x, it->second);
    }

    // Set [l, r) to v, l <= r.
    void assign(long long l, long long r, T v)
    {
        auto R = split(r), L = split(l);
        mp.erase(L, R);
        mp.emplace_hint(R, l, v);
    }

    // f(l, r, value&) for every segment inside [l, r); f may change the value.
    template <class F> void each(long long l, long long r, F f)
    {
        auto R = split(r), L = split(l);
        for (; L != R; ++L) f(L->first, next(L)->first, L->second);
    }
};
```

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　静态模整数

功能：编译期固定模数的取模整数（模数可为合数），支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 `try_inv` 返回空。

核心思路：成员 `v` 始终保存 $[0, \mathrm{mod})$ 内的规范代表，构造时把负余数调正。加减各做一次条件校正，乘法用 `long long` 乘积取模，快速幂按二进制拆分指数。求逆不依赖费马小定理：对 (v, mod) 做迭代扩展欧几里得并维护 v 的系数 `x`，结束时 gcd 为 1 则该系数就是逆元。

接口契约：固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```cpp
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

附录 B　并查集

B.1　并查集

B.1.1　并查集

功能：维护 n 个点的不相交集合，支持合并两个集合、判断两点是否同集合、查询集合大小以及列出所有集合。

核心思路：每个集合是一棵以根为代表的树，`fa` 存父亲，`sz` 在根处存集合大小。`find` 用路径减半：沿父链上走时把每个点的父亲改成祖父。`merge` 找到两根后把 y 的根接到 x 的根下并累加大小。`groups` 按根把点分桶，再删去空桶。

接口契约：dsu(n)，点号 0..n-1，n≥0。find(x) 路径减半；same(x,y) 判断同集合；merge(x,y) 把 y 的根接到 x 的根，已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组，组内点号递增、组间按当前根编号排列；可能压缩路径，但不改变分组。fa、sz 可读，sz 仅根处有效；不要外部改写。本版只做路径减半，不按大小或秩合并；单次 find 最坏 O(n)。空间 O(n)，groups 另需 O(n) 输出与工作空间，并执行 n 次 find。空结构可调用 groups，不能查询顶点。

```
struct dsu
{
    vector<int> fa, sz;

    dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x)
    {
        while (x != fa[x]) x = fa[x] = fa[fa[x]];
        return x;
    }

    bool same(int x, int y) { return find(x) == find(y); }

    bool merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y], fa[y] = x;
        return true;
    }

    int size(int x) { return sz[find(x)]; }

    vector<vector<int>> groups()
    {
        vector<vector<int>> res(fa.size());
        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
        res.erase(
            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
            res.end());
        return res;
    }
};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 1..n，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 67 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3367

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    dsu d(n);
    while (m--)
    {
        int op, x, y;
        scanf("%d%d%d", &op, &x, &y);
        --x, --y;
        if (op == 1)
            d.merge(x, y);
        else
            puts(d.same(x, y) ? "Y" : "N");
    }
}
```

附录 C　树状数组

C.1　树状数组第 k 小与组合依赖

功能：维护长度为 n 的数组，支持单点加、前缀和与区间和；元素表示频数（均非负）时还能求“前缀和首次达到 k 的位置”，即可重集合的第 k 小。

核心思路：a[x] 存原数组在 $(x-\mathrm{lowbit}(x),x]$ 上的和。前缀和沿 $x \leftarrow x-\mathrm{lowbit}(x)$ 累加，单点修改沿 $x \leftarrow x+\mathrm{lowbit}(x)$ 更新所有覆盖它的节点，各 $O(\log n)$。第 k 小从高位到低位倍增：若跳过下一段后累计仍小于 k 就跳过去。

接口契约：保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。n≤100000，|x|≤10000000；总复杂度 O(n log n)，空间 O(n)。

所需模板：Fenwick（第 69 页，C.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

附录 D　线段树

D.1　线段树基础

D.1.1　通用线段树与左右边界搜索

功能：对由幺半群 $(S, \texttt{op}, \texttt{e})$ 元素组成的数组，支持单点赋值、区间按从左到右的顺序合并，以及从一个端点出发求满足单调谓词的最远另一端点。

核心思路：非递归线段树：长度补到 2 的幂 `sz`，叶子在 `d[sz+i]`，内部节点 p 合并 $2p, 2p+1$。`prod` 从两端向上爬，左侧结果 `a` 在右边追加、右侧结果 `b` 在左边追加，保证不交换律也正确。`max_right` 从 l 起每次跳到当前位置所在的最大完整块，若并入后谓词变假，就在该块内向下二分（能并入左孩子则并入并转向右孩子）；`min_left` 对称地从右往左，并始终以 `op(d[r],`a`)` 的顺序合并。

接口契约：segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void set(int p, S x)
    {
        assert(0 <= p && p < n);
        d[p += sz] = x;
        while (p >>= 1) pull(p);
    }

    S get(int p) const
    {
        assert(0 <= p && p < n);
        return d[p + sz];
    }

    S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 71 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

附录 E　树上问题

E.1　树链剖分

E.1.1　重链剖分与 LCA

功能：对一棵树做重链剖分：给出 DFS 序 `dfn`，使每棵子树、每条重链都是连续区间；支持求 LCA，以及把 u–v 路径拆成 $O(\log n)$ 个 `dfn` 区间回调（可只取边对应的位置，也可按 u 到 v 的真实顺序给出），便于套线段树做路径修改与查询。

核心思路：`dfs1` 求父亲、深度、子树大小和重儿子 `son`（子树最大的儿子）；`dfs2` 先递归重儿子，使同一重链的 `dfn` 连续，`top` 为链顶。查询时每次让链顶更深的一端整段跳过本链（上报区间 $[\mathrm{dfn}[\mathrm{top}[u]], \mathrm{dfn}[u]]$）并跳到链顶的父亲，直到两点同链，再上报二者之间的一段；边模式下边权存在深的一端，最后一段跳过较浅点即 LCA。`path_ordered` 中 u 侧的段是自下而上走的，立即以 `rev=true` 回调；v 侧的段先存入 `down`，最后逆序以正向回调。

接口契约：add(u,v) 建无向树；build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 O(log n)，path(u,v,work,edge) 分解路径；work(l,r) 仅适用于可交换聚合，边权模式排除 LCA 位置。输入必须是连通无环树，建树 O(n)，两遍递归 DFS；支持改变根后重复 build，dfn 会重新编号。path_ordered(u, v, work, edge) 的 edge 默认为 false；按真实 u 到 v 顺序回调 work(l, r, rev)：1-based 闭区间 l≤r，rev=true 按 r 到 l 访问，否则按 l 到 r。支持非交换聚合，O(log n) 段与额外空间；结合线段树通常每次查询 $O(\log^2 n)$。edge=true 仅删除 LCA 对应位置，仍须自行处理边算子的方向；不自动求逆。u=v 时点模式一次单点回调，边模式无回调；改变根后须重建依赖 dfn 的结构。

```
struct HLD
{
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, siz, son, top, dfn, rk;

    HLD(int n)
        : n(n),
          g(n + 1),
          fa(n + 1),
          dep(n + 1),
          siz(n + 1),
          son(n + 1),
          top(n + 1),
          dfn(n + 1),
          rk(n + 1)
    {
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[u] = p;
        siz[u] = 1;
        son[u] = 0;
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs1(v, u);
            siz[u] += siz[v];
            if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
        }
    }
```

重编号、LCA 与可交换路径（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        dfn[u] = ++timer;
        rk[timer] = u;
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[u] && v != son[u]) dfs2(v, v);
        }
    }

    // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dep[root] = 0;
        dfs1(root, root);
        dfs2(root, root);
        assert(timer == n);
    }

    int lca(int u, int v) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // Commutative operations only; edge=true excludes the LCA's position.
    template <class F> void path(int u, int v, F work, bool edge = false) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            work(dfn[top[u]], dfn[u]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
    }
```

按真实方向分解路径（接上页同一结构体）：

```
    // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
    template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
    {
        vector<pair<int, int>> down;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                work(dfn[top[u]], dfn[u], true);
                u = fa[top[u]];
            }
            else
            {
                down.emplace_back(dfn[top[v]], dfn[v]);
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
        }
        else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
        for (auto it = down.rbegin(); it != down.rend(); ++it)
            work(it->first, it->second, false);
    }
};
```

模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树，1≤N,Q≤200000。点函数 f(x)=a*x+b，对 998244353 取模；题面 1≤a,c<mod，0≤b,d,x<mod。0 p c d 改单点函数，1 u v x 按 u 到 v（含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间 [l,r] 对应 segtree 半开 [l−1,r)。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改 O(log N)，路径 O($\log^2$ N)，空间 O(N)；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 76 页，E.1.1 节）、segtree（第 71 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/vertex_set_path_composite

```
using S = array<long long, 4>;
const int mod = 998244353;

S op(S a, S b)
{
    return {b[0] * a[0] % mod,
            (b[0] * a[1] + b[1]) % mod,
            a[2] * b[2] % mod,
            (a[2] * b[3] + a[3]) % mod};
}

S e()
{
    return {1, 0, 1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n), val(n);
    for (auto &x : a)
    {
        cin >> x[0] >> x[1];
        x[2] = x[0];
        x[3] = x[1];
    }
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u + 1, v + 1);
    }
    h.build();
    for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
    segtree<S, op, e> s(val);
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(h.dfn[p + 1] - 1, {c, d, c, d});
        }
        else
        {
            int u, v;
            long long x;
            cin >> u >> v >> x;
            h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)
            {
                S z = s.prod(l - 1, r);
                int k = rev ? 2 : 0;
                x = (z[k] * x + z[k + 1]) % mod;
            });
            cout << x << '\n';
        }
    }
}
```

附录 F　最短路问题

F.1　最短路

F.1.1　SPFA：队列松弛、负环判定与有限路径

问题描述：给定可含负权的有向图和起点 s，判断是否存在从 s 可达的负环；若不存在，求 s 到各点的最短距离与一条最短路的原边序列。$s=0$ 时所有点初始距离为 0，可判全图负环，也可直接求差分约束的一组可行解。

核心思路：队列优化的 Bellman–Ford：只有距离被改小的点才入队，再用它松弛出边；`in` 标记点是否已在队中以免重复入队。每次严格松弛时令 `len[v]=len[u]+1`，表示当前标签对应行走的边数；无负环时最短路是简单路径，边数不超过 $n-1$，因此一旦 `len[v]` 达到 n 就立即报告负环。`pre[v]` 记最后更新 v 的边，用于倒推路径。**接口契约：**Spfa(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 返回 0-based 原边号，允许重边、自环、完整 signed64 权值，边数 ≤INT_MAX；加边令 valid=false。run(s) 查从 s 可达的负环，s=0 同时初始化所有点为 0 以查全图。返回 false 时 valid=false，全部 dis/pre 均为未完成状态，不能读取为最短路或恢复路径；需要负环方案或逐点负无穷分类用 BellmanFord。

run 返回 true 时 valid=true，dis[v]==inf 为不可达，其余为 int128 最短距离；run(0) 距离为任意起点出发的最小值，且本身就是全部约束 x_v−x_u≤w 的一个可行解。path(v) 返回正向原边号；不可达与零边路径均为空，结合 dis 区分。每次 run 重置临时队列/标记/路径边数，支持反复改起点与对象复制。

FIFO 队列，最坏 O(nm+n+m) 时间，O(n+m) 空间，不保证近线性；队列去重不等于永久访问标记。严格松弛后维护标签所代表游走的边数，达到 n 立即判负环。停止前标签最多 n 条边，绝对值 ≤n · 2^63<2^94，int128 与 inf=2^126 足够。不要无条件转 long long；e/g/dis/pre/valid 只读，其余按接口使用。

```
struct Spfa
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<long long> dis;
    vector<int> pre;

    Spfa(int n) : n(n), g(n + 1) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        g[u].push_back(int(e.size()) - 1);
        return int(e.size()) - 1;
    }

    // 1-based; s=0 starts from every vertex (difference constraints). False on a negative cycle.
    bool run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
```

```
        vector<int> in(n + 1), len(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; u++)
            if (!s || u == s) dis[u] = 0, in[u] = 1, q.push(u);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int id : g[u])
            {
                int v = e[id].v;
                if (dis[u] + e[id].w >= dis[v]) continue;
                dis[v] = dis[u] + e[id].w, pre[v] = id, len[v] = len[u] + 1;
                if (len[v] >= n) return false;
                if (!in[v]) in[v] = 1, q.push(v);
            }
        }
        return true;
    }

    // Edge ids of the shortest path to t; empty if unreachable or t is a start.
    vector<int> path(int t) const
    {
        vector<int> ids;
        if (dis[t] == inf) return ids;
        for (; pre[t] != -1; t = e[pre[t]].u) ids.push_back(pre[t]);
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：Luogu P3385 / 从 1 可达的负环

题意：T 组图，每条输入边权非负时加双向边，负数仅加 u 到 v；run(1) 只查从 1 可达的负环，有则 YES，否则 NO。m 是输入记录数，不是最终边数；不可达部分的负环不影响本题。

所需模板：Spfa（第 82 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3385

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while (T--)
    {
        int n, m;
        cin >> n >> m;
        Spfa t(n);
        while (m--)
        {
            int u, v, w;
            cin >> u >> v >> w;
            t.add(u, v, w);
            if (w >= 0) t.add(v, u, w);
        }
        cout << (t.run(1) ? "NO" : "YES") << '\n';
    }
}
```

附录 G　生成树问题

G.1　最小生成树

G.1.1　曼哈顿最小生成树

问题描述：给定平面上 n 个点，两点间边权为曼哈顿距离 $|x_i - x_j| + |y_i - y_j|$，求完全图的最小生成树（总权与边集）。

核心思路：每个点只需与它周围 8 个 45° 区域中各自最近的点连边，MST 必在这些候选边中，对称区域合并后共 4 个方向。`candidates` 每次处理一个方向：按 $x + y$ 升序扫描点 i，`sweep` 以 $-y$ 为键保存还没找到最近点的点；取出满足 $y_j \le y_i$ 且 $x_i - x_j \ge y_i - y_j$ 的点 j（即 i 落在 j 的该区域内），由扫描顺序它就是 j 在该区域里最近的点，连边后删去 j。处理完一个方向后交替做交换 x, y 与取负 x 的坐标变换，四次覆盖所有方向，最后对候选边跑 Kruskal。

代码依赖：dsu（第 67 页，B.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Point 为 pair<long long,long long>，坐标支持完整有符号 64 位，点数须 ≤INT_MAX 且内存可分配。candidates(points) 给出最多 4n 条 (weight,u,v) 候选，允许重复边；solve(points) 返回精确 128 位 weight 与原下标边对 edges。空/单点返回 0 且无边，重合点仍为不同顶点；同权最优树不唯一。先转 128 位再变换/加减，输入不变。四方向扫描与 Kruskal 为 O(n log(n+1))、O(n) 空间；调用处选较大 dsu 根在前，不改变普通 dsu 语义。候选保证只针对无约束完整 L1 图，不能直接删禁边后声称仍含受限 MST。

```
struct ManhattanMST
{
    using I = __int128;
    using Point = pair<long long, long long>;
    using Edge = tuple<I, int, int>;
    struct Result
    {
        I weight = 0;
        vector<pair<int, int>> edges;
    };

    static vector<Edge> candidates(const vector<Point> &points)
    {
        assert(points.size() <= INT_MAX);
        int n = int(points.size());
        vector<pair<I, I>> p(points.begin(), points.end());
        vector<int> id(n);
        iota(id.begin(), id.end(), 0);
        vector<Edge> e;
        e.reserve(size_t(n) * 4);
        for (int dir = 0; dir < 4; dir++)
        {
            sort(id.begin(), id.end(), [&](int i, int j)
            {
                return pair{p[i].first + p[i].second, i} <
                       pair{p[j].first + p[j].second, j};
            });
```

活动点扫描与 Kruskal（接上页同一结构体，仍在四轮循环中）：

```
            map<I, int> sweep;
            for (int i : id)
            {
                auto it = sweep.lower_bound(-p[i].second);
                while (it != sweep.end())
                {
                    int j = it->second;
                    I dx = p[i].first - p[j].first;
                    I dy = p[i].second - p[j].second;
                    if (dx < dy) break;
                    e.emplace_back(dx + dy, i, j);
                    it = sweep.erase(it);
                }
                sweep[-p[i].second] = i;
            }
            for (auto &[x, y] : p)
                if (dir & 1) x = -x;
                else swap(x, y);
        }
        return e;
    }

    static Result solve(const vector<Point> &points)
    {
        auto e = candidates(points);
        int n = int(points.size());
        Result r;
        if (n < 2) return r;
        sort(e.begin(), e.end());
        dsu uf(n);
        for (auto [w, u, v] : e)
        {
            int a = uf.find(u), b = uf.find(v);
            if (a == b) continue;
            if (uf.size(a) < uf.size(b)) swap(a, b);
            uf.merge(a, b);
            r.weight += w;
            r.edges.emplace_back(u, v);
            if (r.edges.size() == points.size() - 1) break;
        }
        assert(r.edges.size() == points.size() - 1);
        return r;
    }
};
```

模板题使用：Library Checker Manhattan MST

题意：1≤N≤200000，N 个点的坐标 0..10^9，可重合且每个输入位置仍是不同顶点。完整图边权为曼哈顿距离；输出最小总权及 N−1 条 0-based 原编号边，任意最优树均可。结果 weight 为 128 位；本题粗上界 2e9 乘以 N−1，严格小于 4e14，所以此驱动才可转 long long 打印。N=1 输出 0、不输出边；库的空输入和有符号 64 位坐标另属 API 扩展。

所需模板：dsu（第 67 页，B.1.1 节）、ManhattanMST（第 85 页，G.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/manhattanmst

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<ManhattanMST::Point> p(n);
    for (auto &[x, y] : p) cin >> x >> y;
    auto r = ManhattanMST::solve(p);
    cout << (long long)r.weight << '\n';
    for (auto [u, v] : r.edges) cout << u << ' ' << v << '\n';
}
```

G.1.2 Kruskal 最小生成树与生成森林

问题描述：给定无向带权图，求最小生成树；图不连通时求最小生成森林（每个连通块一棵最小生成树），给出总权、所选边号和连通块数。

核心思路：把所有边按权值升序稳定排序（同权按编号），依次考察：若两端点在并查集中不连通，就选这条边并合并，否则跳过（会成环）。每选一条边连通块数减一，因此最终 `cnt` 就是连通块数，为 1 表示整图连通、得到的是最小生成树。

代码依赖：dsu（第 67 页，B.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Kruskal(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 加入无向边并返回 0-based 原边号，边权为完整 signed64，支持重边、自环和负权，边数 ≤INT_MAX。run() 每次从全部原边重建最小生成森林，返回连通块数；n>0 时返回 1 表示存在 MST，空图返回 0。weight 为 int128 总权，ids 为所选原边号，按 (权值, 原编号) 排序；e 保持插入顺序。加边后必须重新 run，旧结果不代表新图。支持重复运行和独立值复制。

依赖普通 dsu，调用方先把较大根放在前，保留 dsu 的 fa[y]=x 语义。时间 O(n+m log(m+1)+m α(n+1))，空间 O(n+m)；add 均摊 O(1)。总权绝对值 <2^94，不能无条件转 long long。自环不入树，断开图仍返回各块最小树；不使用 −1 总权判断失败，合法 MST 总权也可为 −1。成员只读，n 和 e 不直接修改。

```
struct Kruskal
{
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<int> ids;
    long long weight = 0;

    Kruskal(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // Minimum spanning forest on 0..n-1; ties by edge id. Returns the component count.
    int run()
    {
        vector<int> order(e.size());
        iota(order.begin(), order.end(), 0);
        stable_sort(order.begin(), order.end(), [&](int a, int b) { return e[a].w < e[b].w; });
        dsu d(n);
        ids.clear();
        weight = 0;
        int cnt = n;
        for (int id : order)
```

```
32              if (d.merge(e[id].u, e[id].v))
33              {
34                  ids.push_back(id);
35                  weight += e[id].w;
36                  cnt--;
37              }
38          return cnt;
39      }
40  };
```

模板题使用：Luogu P3366 / 最小生成树

题意：无向图求最小生成树总权，不连通输出 orz；n≤5000、m≤200000、边权 1..10000，允许自环重边。输入 1-based 点号减一，run()!=1 表示断连；本题总权可安全转 long long。

所需模板：dsu（第 67 页，B.1.1 节）、Kruskal（第 88 页，G.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3366

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Kruskal g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    if (g.run() != 1)
    {
        cout << "orz\n";
    }
    else
    {
        cout << (long long)g.weight << '\n';
    }
}
```

算法索引

Dinkelbach, 57
dinkelbach, 57
Dominance3D, 32
dsu, 67
DynamicDP, 18

ExactCover, 1
Expression, 58

Fenwick, 69

grid_cycle, 9
grid_cycles, 14

HLD, 76

Knuth 优化, 27
knuth_interval_dp, 29
Kruskal, 88

manhattan_order, 39
ManhattanMST, 85
MinimumCover, 5
Mo, 39
ModifiedMo, 42
ModInt, 63
Monge, 25
monotone_dp_layer, 26
MonotoneHull, 22

ODT, 61
OverallKth, 35

quadrangle inequality, 25

RollbackMo, 47

segtree, 71
Spfa, 82

TreeMo, 45

wqs_independent_set, 31

xor_hamming_pairs, 52

决策单调性, 25
    分治, 25

区间, 27
分数规划, 57
分治优化, 25
四边形不等式, 25
带修莫队, 45